UNIVERSITY OF LATVIA
FACULTY OF SCIENCE AND TECHNOLOGIES

Comparison system for virtual fitting data used in the garment models' construction process

QUALIFICATION PROJECT

Author: Aleks Ilves
Student's ID card No.: ai23031

Advisor: Dipl.-Math. (WWU Münster) Andrejs Neimanis

Riga 2026

ABSTRACT

The “Comparison system for virtual fitting data used in the garment models’ construction process” is designed to analyse and compare how garments fit on different 3D body models by using advanced computational geometry and marker-based distance evaluation. The system processes virtual body scans with anatomical markers and corresponding garment meshes to calculate fit metrics using multiple ray-casting, nearest-point, and hybrid distance-measurement algorithms. In addition, the system enables enhanced local volume analysis and similarity scoring to assess structural fit differences across models more accurately. The software provides tools for evaluating garment-body interactions, calculating similarity percentages between virtual fits, and generating heatmap visualizations for intuitive interpretation. The document outlines the system architecture, software requirements, algorithmic design, functional capabilities, comparison methodology, and visualization components. The document is developed according to the ISO/IEC/IEEE 29148:2018 standard.

Keywords: virtual fitting, garment simulation, 3D body modelling, fit comparison, computational geometry, similarity analysis

ANOTĀCIJA

"Apģērbu modeļu konstruēšanas procesā izmāntoto virtuālās uzlaikošanas datu salīdzināšanas sistēma", ir izstrādāta, lai analizētu un salīdzinātu, kā apģērbi der dažādiem 3D ķermeņa modeļiem, izmantojot progresīvu skaitļošanas ģeometriju un marķieru balstītu attāluma noteikšanu. Sistēma apstrādā virtuālos ķermeņa skenējumus ar anatomiskajiem marķieriem un atbilstošajiem apģērbu attēlojumiem, lai aprēķinātu piemērotības rādītājus, izmantojot vairākus staru izkliedēšanas, tuvākā punkta un hibrīda attāluma mērīšanas algoritmus. Turklāt sistēma ļauj veikt uzlabotu lokālā tilpuma analīzi un līdzības novērtēšanu, lai precīzāk novērtētu strukturālās piemērotības atšķirības starp apģērbu modeļiem. Programmatūra nodrošina rīkus apģērba un ķermeņa mijiedarbības novērtēšanai, virtuālo piemērotību līdzības procentu aprēķināšanai un ļauj ģenerēt siltuma karšu vizualizāciju intuitīvai interpretācijai. Dokumentā ir izklāstīta sistēmas arhitektūra, programmatūras prasības, algoritmu dizains, funkcionālās iespējas, salīdzināšanas metodika un vizualizācijas komponenti. Dokuments ir izstrādāts saskaņā ar standartu ISO/IEC/IEEE 29148:2018.

Atslēgvārdi: virtuālā pielaikošana, apģērba simulācija, 3D ķermeņa modelēšana, pielaikošanas salīdzinājums, aprēķinu ģeometrija, līdzības analīze

TABLE OF CONTENTS

LIST OF FIGURES 5
GLOSSARY 7
INTRODUCTION 9
Purpose 9
Scope 9
Overview 9
1. OVERALL DESCRIPTION 11
1.1 AS IS 12
1.2 Client 13
1.3 Product Vision 13
1.4 Business Requirements 13
1.5 System Users 14
1.6 General restrictions/limitations 15
1.7 Assumptions and Dependencies 15
2. SOFTWARE REQUIRMENTS SPECIFICATION 16
2.1 Conceptual Database Model 16
2.2 Functional Requirements 18
2.2.1 General Description of functions 18
2.2.2 Functional breakdown by modules 20
2.2.3 User & Dataset Management Module 22
2.2.4 Body & Garment Processing Module 26
2.2.5 Analysis Execution Module 31
2.2.6 Distance & Fit Calculation Module 35
2.2.7 Comparison Reporting Module 40
3. SOFTWARE DESIGN DESCRIPTION 45
3.1 Database Design 45
3.1.1 Logical Model 45
3.1.2 Physical Model 46
3.1.3 Database Entity description 47
3.2 Design of User Interface 53
3.2.1 Login window 53
3.2.2 Main analysis window 53

3.2.3 Visualization window 54
3.2.4 Analysis history window 56
3.3 Functional Design 58
3.3.1 Class Diagram 58
3.3.2 Algorithm flowcharts 59
3.3.2.1 User Authentication algorithm 59
3.3.2.2 Analysis Configuration algorithm 59
3.3.2.3 Model Loading algorithm 60
3.3.2.3 Fit Distance Calculation algorithm 60
3.3.2.4 Enhanced Volume Analysis algorithm 61
3.3.2.5 Fit Comparison algorithm 61
3.3.2.6 Result Visualization algorithm 62
3.3.2.7 Analysis History Management algorithm 62
4. TESTING 63
4.1 Testing Plan 63
4.1.1 Scope and objectives 63
4.1.2 Selected Framework 63
4.2 Test Journal 65
4.2.1 Test statement coverage and decision coverage 65
4.2.2 Test scope and test levels 66
4.2.3 Test conclusions 67
4.2.3.1 Discovered Bugs 68
4.2.4 Test Runs 68
5. PROJECT MANAGEMENT 69
5.1 Project organization 69
5.2 Research 71
5.3 Version control 73
5.4 Effort estimation 74
APPENDIX 77
Coding standards 77
Coding standard exceptions 79
REFERENCES AND SOURCES 80

LIST OF FIGURES

Number	Title
1.	Conceptual database model (Crowfoot syntax)
2.	Modules of the system
3.	Data flow diagram (Level 0)
4.	Data flow diagram (Level 1)
5.	User & Dataset Management module diagram
6.	Body & Garment Processing module
7.	Analysis Execution model
8.	Distance & Fit Calculation module
9.	Comparison Reporting module
10.	Logical Model Diagram
11.	Physical Model Diagram
12.	Login window
13.	Main window
14.	Side by side visualization
15.	Heatmap visualization
16.	Garment similarity visualization
17.	History window
18.	History window (admin view)
19.	System class diagram
20.	User Authentication algorithm
21.	Analysis Configuration algorithm
22.	Model Loading algorithm
23.	Fit Distance Calculation algorithm
24.	Enhanced Volume Analysis algorithm
25.	Fit Comparison algorithm
26.	Result Visualization algorithm
27.	Analysis History Management algorithm

28.	Test execution
29.	Code metrics result

GLOSSARY

Term	Definition
Body Marker / Marker / Topcode Marker	A Body Marker is a predefined anatomical reference point embedded in a 3D body model and represented as a vertex with associated semantic meaning. In the implemented system, body markers are identified by structured comments in the source OBJ file (e.g. # topcode_marker <id>) and are assumed to be unique within a body model. Body markers serve as stable reference points for quantitative analysis by defining locations of interest on the body surface (such as shoulder, chest, or armhole regions). Distances between these markers and the garment surface are computed to enable localized fit assessment and comparison across different body models.
3D Model	A digital representation of a physical object defined by vertices, edges, and faces in three-dimensional space. In this project, 3D models are used to represent human bodies and garments.
OBJ (Wavefront OBJ)	A widely used text-based file format for representing 3D geometry, storing vertex coordinates, face definitions, and optional comments. The project uses OBJ files to store body models, garment models, and embedded body markers.
Body Model	A 3D representation of a human body surface used as the reference geometry for garment fit analysis. Body models contain embedded anatomical markers that define specific regions of interest.
Garment Model	A 3D mesh representing a garment, positioned around a body model. The garment model is analyzed relative to the body surface to compute geometric fit indicators.
Marker-Based Analysis	An analysis approach that computes garment-body distances at predefined anatomical reference points (body markers) instead of relying on dense per-vertex comparisons.
Geometric Distance	A numerical measure of spatial separation between two geometric entities. In this system, distances are computed between body markers or body surface points and the garment surface

Ray Casting	A geometric technique in which a ray is projected from a point in a specified direction to determine intersections with a surface. Used in this project to compute garment distances from body markers.
KD-Tree (k-dimensional Tree)	A spatial data structure used for efficient nearest-neighbor searches in multidimensional space. In this project, KD-Trees are used to find the closest garment vertex to a given body marker or surface point.
Hybrid Distance Method	A distance calculation approach that combines multiple techniques (normal-based ray casting, centroid-based ray casting, and nearest-neighbor search) to improve robustness when individual methods fail.
Local Volume Analysis	An optional analysis method that estimates the local volumetric relationship between the body and garment around a marker by computing convex hull volumes within a defined radius.
Similarity Percentage	A normalized metric expressing how similar two garment fit scenarios are, based on aggregated geometric differences between corresponding markers or surface regions.

INTRODUCTION

Purpose

This document describes the “Comparison system for virtual fitting data used in the garment models’ construction process,” including the system’s vision, intended users, core functionality, computational methods, design principles, assumptions, limitations, and user interface elements. It provides a detailed specification of the algorithms used for garment-body fit analysis, data processing workflows, similarity computation, and visualization tools. The document also outlines the software requirements and design specifications necessary for developing and evaluating the system.

Scope

The comparison system enables users to analyse, compare, and interpret virtual fitting data generated for 3D body models in garment construction workflows. The system accepts virtual body scans with anatomical markers and corresponding garment meshes, performs distance- and intersection-based measurements using multiple comparison algorithms, and generates quantitative similarity metrics and visual heatmaps.

It supports apparel researchers, pattern designers, and virtual fitting specialists who need precise, reproducible evaluations of garment fit across different body models. The system automates geometric calculations, provides multi-algorithm comparison options, and ensures consistent analysis for model validation and garment-fit assessment.

Administrators and technical users can configure algorithm parameters, manage datasets, and customize output formats. The system is designed to be extendable, allowing future integration with simulation software, CAD tools, and fit-prediction pipelines.

Overview

The document is divided into several chapters, each covering a specific aspect of the system:

1. Overall Description - Provides the product vision, intended users, use cases, and general constraints of the system. This chapter also introduces the core concepts of virtual fitting, marker-based body models, garment meshes, and fit comparison requirements.

2. Software Requirements Specification - Details the functional and non-functional requirements of the comparison system. It includes module-level descriptions of the distance-measurement algorithms, data import and preprocessing routines, similarity-scoring components, and visualization tools. Data flow diagrams, structural diagrams, and functional descriptions are provided.
3. Software Design Description - Presents the architecture and implementation design of the system. It explains how comparison algorithms are structured in the codebase, describes data models and file-processing logic, and includes UI mock-ups for visualization panels and reporting modules.
4. Testing - Defines the scope of testing, test cases, expected outputs, algorithm validation strategies, and overall test coverage. This chapter verifies the system's accuracy, reliability, and performance across multiple fitting scenarios.
5. Effort Estimation - Summarizes the distribution of time spent on development, research, algorithm design, dataset preparation, documentation, and testing.

1. OVERALL DESCRIPTION

Modern apparel development increasingly relies on virtual fitting technologies to evaluate garment performance before physical prototyping. Historically, garment construction has been performed in two-dimensional pattern space, with early virtual fitting tools serving mainly as approximate visualizations of how a garment might appear when worn. Over time, advances in simulation and visualization enabled these tools to be used not only for appearance preview, but also as indicators of garment fit quality through fitting maps such as distance, strain, and stress visualizations.

More recently, garment construction workflows have shifted toward three-dimensional environments, where garments are directly manipulated on 3D body models and patterns are recalculated automatically from the 3D design. Despite this progress, virtual fitting processes remain largely oriented toward standard garment sizes and a small number of reference avatars. Individualized or algorithmically generated garments-where body measurements directly influence garment geometry-introduce a fundamental challenge: it is infeasible to manually inspect or validate fit quality across hundreds of diverse body shapes.

Furthermore, virtual fitting maps provide informative but non-absolute visual indicators. They are not photo-realistic representations of real-world fit and cannot fully replace physical fitting and sewing. Their primary value lies in comparative analysis rather than absolute judgment. When applied at scale, the key challenge is no longer generating fitting maps, but interpreting them consistently and identifying cases where garment fit is likely to be problematic.

The core idea of this project is to support automated, quantitative comparison of virtual fitting results across many body models. Instead of evaluating fit subjectively or visually for each individual case, the system focuses on identifying relative differences and similarities between garment-body configurations. To achieve this, the comparison system processes 3D body models with anatomical markers alongside corresponding garment meshes. Using geometric algorithms such as ray casting, nearest-point analysis, hybrid distance measurements, and optional local volume estimation, the system quantifies spatial relationships between the garment and the body surface. The resulting numerical indicators allow the comparison of fitting behavior across different avatars, independent of visual appearance.

By automating complex geometric analysis and enabling reproducible comparisons, the system supports large-scale virtual fitting evaluation workflows. It provides a foundation for identifying critical body types, validating individualized garment constructions, and supporting data-driven decision-making in digital garment development and research contexts.

1.1 AS IS

Current virtual fitting processes in the apparel industry rely on advanced 3D garment construction and visualization systems that are primarily designed for standard-size garment development and interactive design workflows. Tools such as CLO3D, Browzwear VStitcher, Tuka3D, Assyst/3D Vidya, and Marvelous Designer offer realistic visualization, fabric simulation, and fitting map overlays, enabling designers to assess garment behavior on a small number of reference avatars.

However, these systems exhibit several limitations when applied to research-oriented or individualized garment development scenarios.

First, most commercial tools present fitting information mainly as visual overlays, without providing access to the underlying numerical data used to generate these maps. This limits the possibility of performing quantitative comparison, statistical analysis, or automated evaluation across multiple fitting scenarios-capabilities that are essential when studying body-shape diversity or individualized clothing.

Second, existing virtual fitting environments are optimized for manual, interactive workflows, such as on-screen garment adjustment, manual sewing, and single-avatar evaluation. While effective for design iteration, these workflows do not scale to large datasets involving tens or hundreds of body models, which are increasingly relevant in anthropometric research and mass customization contexts.

Third, although some systems expose limited APIs, access to comprehensive fitting metrics is typically restricted or proprietary. Blender is currently the only widely available tool that provides programmatic access to several fitting-related numerical maps (e.g., distance, strain, stretch, shear), making it suitable for transparent and reproducible quantitative analysis.

As a result, there is currently no lightweight, tool-independent solution that enables automated, quantitative comparison of virtual fitting results across large numbers of body models. This gap motivates the development of a standalone analysis pipeline, such as the one implemented in this

project, which focuses on geometric comparison metrics and operates independently of closed, proprietary garment simulation environments.

1.2 Client

The system is developed as a qualification project by a student and is intended for use by educational institutions, research laboratories, or companies working in virtual garment simulation, apparel engineering, and computational geometry.

Potential clients include fashion technology firms, garment manufacturers, simulation software developers, and academic groups conducting research on virtual fitting accuracy and garment-body interaction.

1.3 Product Vision

The system aims to provide a scientific and standardized method for evaluating and comparing garment fit on virtual body models. Users will be able to upload body scan models with markers; process garment meshes and apply various comparison algorithms to extract quantitative information about garment fit.

The product provides:

- Precise geometric measurements between garment surfaces and body markers
- Multiple comparison algorithms suitable for different garment types and scenarios
- Visualization of fit differences through distance colour maps and structured reports
- Similarity scoring to determine how closely different models' fittings match
- Tools to support decision-making in garment pattern development and virtual model validation

  The system is not intended to replace full garment simulation software but to augment existing digital workflows with accurate comparison and measurement functionality. It serves as an analytical tool that improves reliability and reproducibility in virtual fitting evaluation.

1.4 Business Requirements

The system is developed as a technical research tool intended for processing and comparing 3D body and garment models using quantitative geometric metrics. It supports research activities

related to virtual fitting, anthropometric data analysis, and evaluation of garment-to-body deviations. The system must satisfy the following requirements:

1. The system shall allow users to upload 3D body and garment files in supported formats (e.g., OBJ) for further processing.
2. The system shall extract and process body markers from the uploaded body model to enable quantitative measurement.
3. The system shall compute geometric distance-based indicators, such as minimum, maximum, and average point-to-surface distances.
4. The system shall support configurable analysis parameters, including distance calculation methods and optional volume-based similarity estimation.
5. The system shall provide a structured analysis result, presenting numerical indicators that describe geometric differences between the body and garment models.
6. The system shall allow exporting analysis results in a simple, machine-readable format to support further research, comparison, or visualization.
7. The system shall not perform fabric simulation, physical draping, or any subjective fit assessment, but shall strictly operate on geometric data.
8. The system shall support reproducible, repeated processing, allowing the same configuration to be applied consistently to multiple datasets.
9. The system shall operate independently of proprietary garment simulation tools, enabling researchers to work with openly accessible data and processing pipelines.

1.5 System Users

The system includes three types of users:

1. General Users

   Apparel designers, simulation specialists, and researchers who analyse garment-body fit. They can upload datasets, run comparison algorithms, and view or export results.
2. Advanced Users / Technical Operators

   Users with deeper technical knowledge, responsible for configuring algorithm parameters, managing datasets, and ensuring workflows operate correctly.

3. Administrators

   Maintain system settings, manage user access, and oversee data storage. They have full permissions and may also be general users.

1.6 General restrictions/limitations

- Users must have access to a computer capable of processing 3D models and performing geometric calculations.
- The system requires valid 3D body models and garment meshes prepared according to the expected format and marker structure.
- Complex or extremely high-resolution meshes may affect performance or processing time.
- The system does not generate garment simulations; it only analyses pre-simulated or pre-generated virtual fitting data.
- Data storage and processing must comply with institutional and regional data protection regulations where applicable.

1.7 Assumptions and Dependencies

- The system assumes that all uploaded 3D body models include correctly placed anatomical markers.
- Garment meshes are assumed to originate from external simulation software and must accurately represent garment surfaces.
- The accuracy of comparison results depends on the quality of input models and simulation data.
- The system relies on established computational geometry libraries and 3D processing frameworks to support its algorithms.
- Users are assumed to possess basic understanding of 3D garment fitting concepts and digital modelling workflows.

2. SOFTWARE REQUIRMENTS SPECIFICATION

2.1 Conceptual Database Model

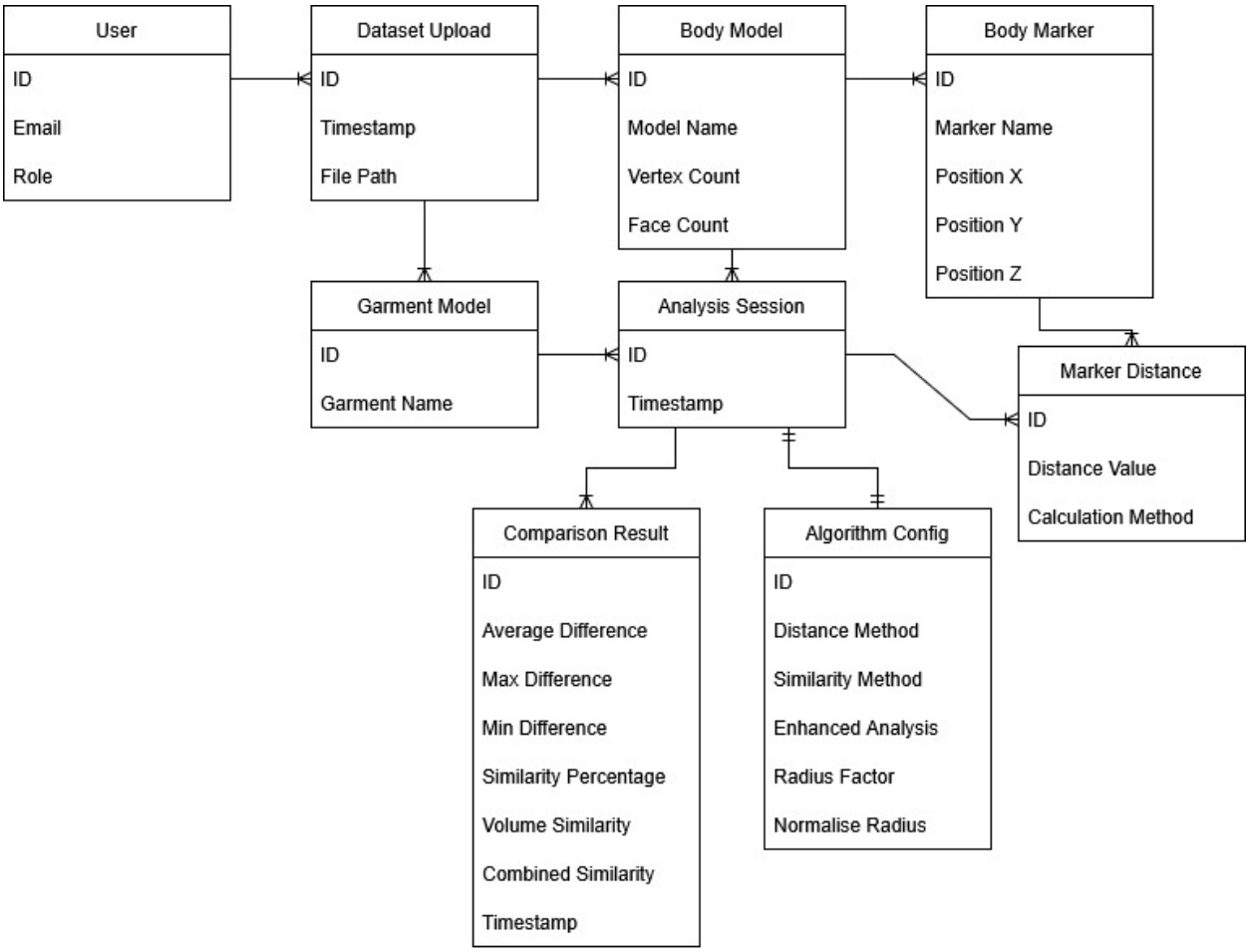


Fig 1. Conceptual database model (Crowfoot syntax)

- Entity “User” represents a person interacting with the system, such as a researcher or developer conducting virtual fitting analyses. It stores basic identification and access-related information, including the User ID, Email, and Role. A single user may work with multiple datasets over time; therefore, the “User” entity has a one-to-many relationship with the “Dataset Upload” entity.
- Entity “Dataset Upload” represents a single upload event initiated by a user. In practice, this corresponds to a logical grouping of input data that is intended to be analysed together, such as a set of body models and garment models used in one experimental scenario. The entity stores the upload identifier, timestamp, and file path information.

One dataset upload may contain multiple body models and multiple garment models, which is reflected in its one-to-many relationships with the “Body Model” and “Garment Model” entities.

- Entity “Body Model” represents a three-dimensional digital human body mesh used for virtual fitting analysis. In addition to basic metadata such as Model Name, Vertex Count, and Face Count, the body model serves as the geometric reference against which garments are evaluated. Each body model may be reused across multiple experiments, which is why it has a one-to-many relationship with the “Analysis Session” entity. Furthermore, each body model contains a set of anatomical reference points, represented by the “Body Marker” entity.
- Entity “Body Marker” represents a predefined anatomical reference point on the body surface, defined by a specific vertex position (X, Y, Z). In practice, body markers are used to identify meaningful regions of interest on the human body, such as the shoulder, chest, waist, or armhole areas, where garment fit is especially critical.
  Although the software is technically capable of working with any set of markers, in realistic use cases these markers are assumed to be uniquely identifiable and consistently placed across different body models. This allows meaningful comparison of corresponding regions between different bodies and garments. Each body marker belongs to exactly one body model and can generate multiple distance measurements during analysis, resulting in a one-to-many relationship with the “Marker Distance” entity.
- Entity “Garment Model” represents a three-dimensional garment mesh that is evaluated against one or more body models. The entity stores identifying information such as the garment ID and name. Each garment model may be reused across multiple experiments, which is why it has a one-to-many relationship with the “Analysis Session” entity.
- Entity “Analysis Session” represents a single execution of the garment-body comparison process. Conceptually, it corresponds to one analytical experiment where a specific body model and garment model are evaluated using a defined set of algorithm parameters. The entity stores the session identifier and timestamp.
  An analysis session produces multiple detailed distance measurements (per body marker) and may also produce one or more aggregated comparison results. For this reason, it has one-to-many relationships with both the “Marker Distance” and “Comparison Result”

entities. Each analysis session is also associated with exactly one “Algorithm Config”, ensuring that the analysis is fully reproducible.

- Entity “Marker Distance” represents an individual quantitative measurement of the geometric distance between a specific body marker and the garment surface. It stores the calculated distance value and the calculation method used. Each marker distance belongs to exactly one analysis session and one body marker, reflecting its role as a fine-grained measurement used to assess local garment fit.
- Entity “Comparison Result” represents an aggregated outcome derived from an analysis session. It stores summary metrics such as average, minimum, and maximum distance differences, similarity percentages, and optional volume-based similarity indicators. These results provide a concise numerical description of how similarly a garment fits across different body models. Each comparison result is associated with exactly one analysis session.
- Entity “Algorithm Config” represents the configuration of algorithmic parameters used during an analysis session. This includes the selected distance calculation method, similarity method, optional enhanced analysis flag, radius factor, and radius normalization setting. By storing this information explicitly, the system ensures transparency and reproducibility of analytical results. Each algorithm configuration is linked one-to-one with an analysis session.

2.2 Functional Requirements

2.2.1 General Description of functions

The system consists of 5 main modules: User & Dataset Management Module, Body & Garment Processing Module, Analysis Execution Module, Distance & Fit Calculation Module, and Comparison Reporting Module.

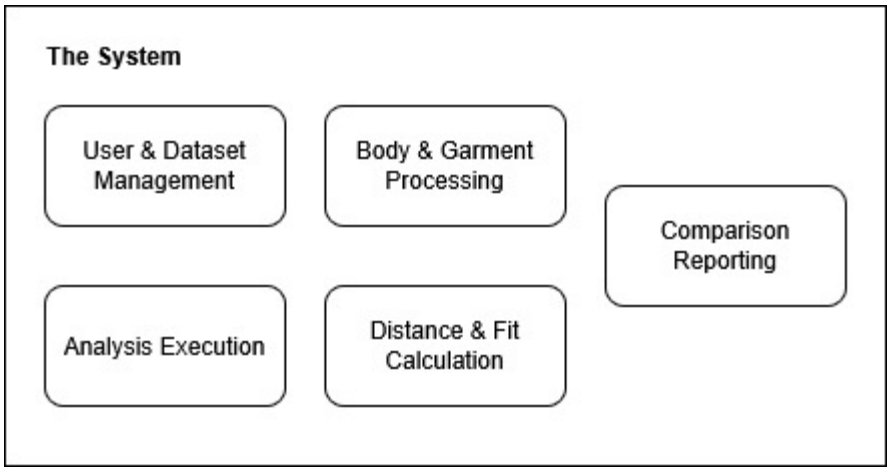


Fig 2. Modules of the system

Module	Function	User Group
User & Dataset Management Module	Upload dataset files (body or garment)	User, Admin
	View upload history	User, Admin
	Delete uploaded dataset	Admin
	Manage user roles	Admin
Body & Garment Processing Module	Load body model file	User, Admin
	Load garment model file	User, Admin
	Extract body markers	System
	Validate model geometry	System / Admin
	Display model data (vertex count, face count)	User, Admin
Analysis Execution Module	Create analysis session	User, Admin
	Configure algorithm parameters	User, Admin
	Start analysis (distance & similarity computation)	User, Admin
	View session details	User, Admin
	Delete analysis session	Admin

Distance & Fit Calculation Module	Calculate per-marker distance	System
	Apply selected distance method	System
	Generate enhanced analysis output (when enabled)	System
	Store marker-distance results	System
Comparison Reporting Module	Generate comparison result metrics	System
	Display average / max / min differences	User, Admin
	Display similarity percentages (volume, combined)	User, Admin
	Export comparison results	User, Admin
	View result history	User, Admin

Table 1. Module functions and corresponding user groups

2.2.2 Functional breakdown by modules

The data flow diagrams (Fig.3 and Fig.4) display functional breakdown by modules based on the description in Table 1 and Fig.2.

Several modules are interconnected and rely on data from other modules in order to function properly. These modules are:

- **Dataset Management** is responsible for receiving and storing uploaded files, and it provides the processed dataset metadata to the Model Processing module. Without dataset information, the system cannot extract body models, garment models, or prepare data for further analysis.
- **Model Processing** depends on dataset uploads to build body models, garment models, and body markers. The extracted results are then used by the Analysis Session

Management module, which requires body and garment data to initialize a valid analysis session.

- **Analysis Session Management** relies on both the Model Processing and Dataset Management modules because it needs body model data, garment model data, and algorithm configuration parameters. The output of this module is used directly by the Distance Calculation Management module, which cannot perform any calculations without a valid session.
- **Distance Calculation Management** depends on body markers, garment mesh data, and active analysis session details to compute per-marker distances. This module provides essential measurement data to the Comparison Result Management module, which requires these distance values to aggregate analysis results.
- **Comparison Result Management** relies on outputs from both the Analysis Session Management and Distance Calculation Management modules. It uses stored marker distances and session information to generate overall similarity metrics; compute combined similarity and deliver the final report back to the user or administrator.

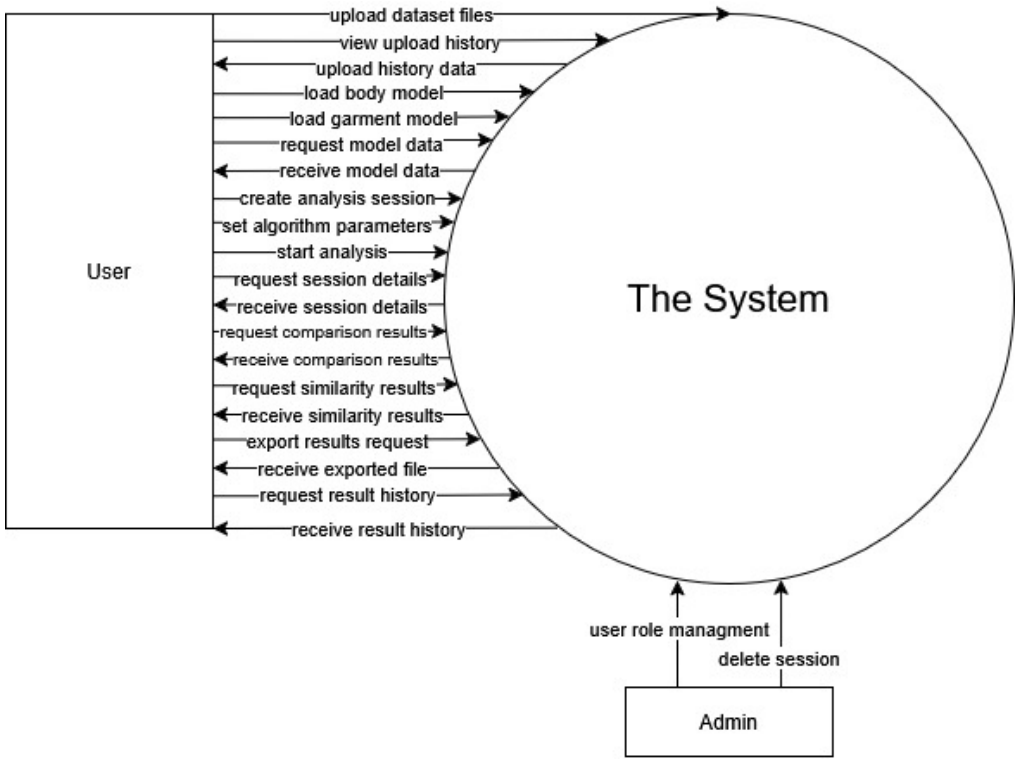


Fig 3. Data flow diagram (Level 0)

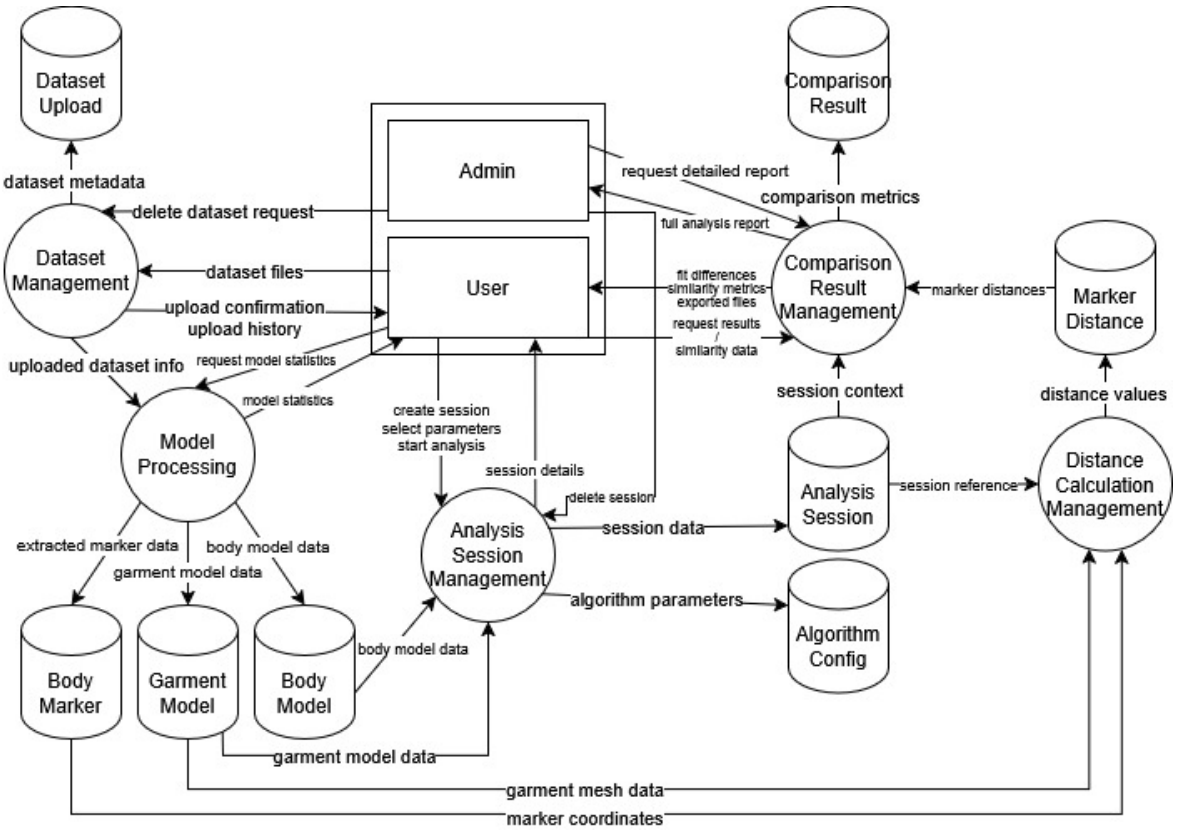


Fig 4. Data flow diagram (Level 1)

2.2.3 User & Dataset Management Module

Users interact with this module to upload and manage their dataset files, which serve as the foundation for all later processing stages in the system. The module allows Users and Administrators to upload body and garment model files (e.g., .obj format), validate their structure, store dataset metadata, and view previously uploaded datasets. The stored datasets become available for the Body & Garment Processing Module, where further model extraction and analysis occur. This module ensures that all uploaded data is valid, stored correctly, and accessible for downstream operations.

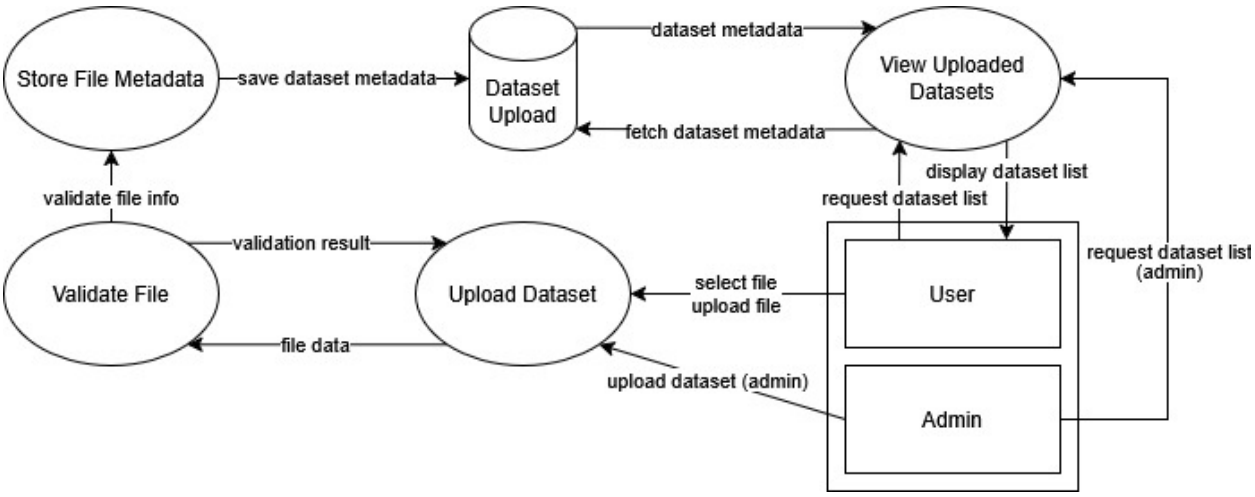


Fig 5. User & Dataset Management module diagram

USER.DATASET.UPLOAD

As a User, I want to upload my body and garment dataset files so that they can be used in future analysis sessions.

Actor	Description
User	A general system user.
Admin	A privileged user with extended management capabilities.

Pre-conditions
1. The User is authenticated in the system. 2. The User has valid dataset files containing body or garment model information (.obj).

Post-conditions
The following conditions must be met when this use case is completed: 1. The dataset file is successfully uploaded and stored. 2. The system records metadata such as file path, timestamp, and dataset type. 3. The User can view the uploaded dataset in their dataset list.

Flow of Events
1. The User navigates to the “Upload Dataset” section.

2. The User selects the dataset file from their device.
3. The User presses the “Upload” button.
4. The system receives the file and performs initial validation.
   4.1. **Alternative A1**: The file format is invalid.
5. The system extracts basic metadata (file size, name, path, timestamp).
6. The system saves the dataset metadata in the Dataset Upload data store.
7. The system confirms that the upload is successful.
8. The dataset becomes available for future body/garment model extraction.

The use case ends.

Alternative flows:

A1: Invalid file format
1. The system displays an error message: “Invalid file type. Only .obj files are supported.” 2. The User selects a new file and attempts upload again. 2.1. **Alternative A1** repeats if invalid again. The alternative flow ends, basic flow continues at step 3.

USER.DATASET.VIEW

As a User, I want to view all the datasets I have uploaded so that I can track, reuse, or verify them before performing any analysis.

Actor	**Description**
User	A general user system user.
Admin	A privileged user; can view all uploaded datasets.

Pre-conditions
1. The User is authenticated. 2. At least one dataset exists, uploaded by the User or Administrator.

Post-conditions

The following conditions must be met when this use case is completed: 1. The User views a list of all previously uploaded datasets. 2. Metadata displayed includes: file name, model type, timestamp, and file path.

Flow of Events
1. The User navigates to the “My Datasets” page. 2. The system requests dataset metadata from the Dataset Upload data store. 3. The dataset list is displayed to the User. 3.1. **Alternative A1**: No datasets exist. 4. The User may click on any dataset to view additional details. The use case ends.

Alternative flows:

A1: No datasets available
1. The system displays the message: “No datasets uploaded yet.” 2. The User presses “Upload Dataset” and is redirected to the upload page. The alternative flow ends.

USER.DATASET.DELETE

As an Administrator, I want to remove invalid or outdated datasets to ensure system integrity.

Actor	**Description**
Admin	A privileged user; can remove irrelevant datasets.

Pre-conditions
1. The Administrator is authenticated. 2. The dataset exists in the system.

Post-conditions
The following conditions must be met when this use case is completed:

1. The dataset record is removed from the Dataset Upload store. 2. The dataset is no longer available for analysis.

Flow of Events
1. The Administrator navigates to the Dataset Management section. 2. The Administrator selects a dataset and presses “Delete”. 3. The system requests confirmation. 4. The Administrator confirms deletion. 5. The system removes the dataset metadata from the database. 6. The system displays a success message. The use case ends.

2.2.4 Body & Garment Processing Module

The Body & Garment Processing Module is responsible for extracting, interpreting, and preparing 3D model data for analysis. Once a dataset has been uploaded, this module allows the User or Administrator to load the corresponding body and garment model files, parse their geometric structure, and extract essential components such as vertices, faces, and body markers. Additionally, the module assigns models to their originating dataset and prepares them for use in later stages of the system, including analysis session creation, distance calculations, and comparison generation.

This module ensures that uploaded 3D files are correctly formatted, readable, and structurally valid before analysis begins.

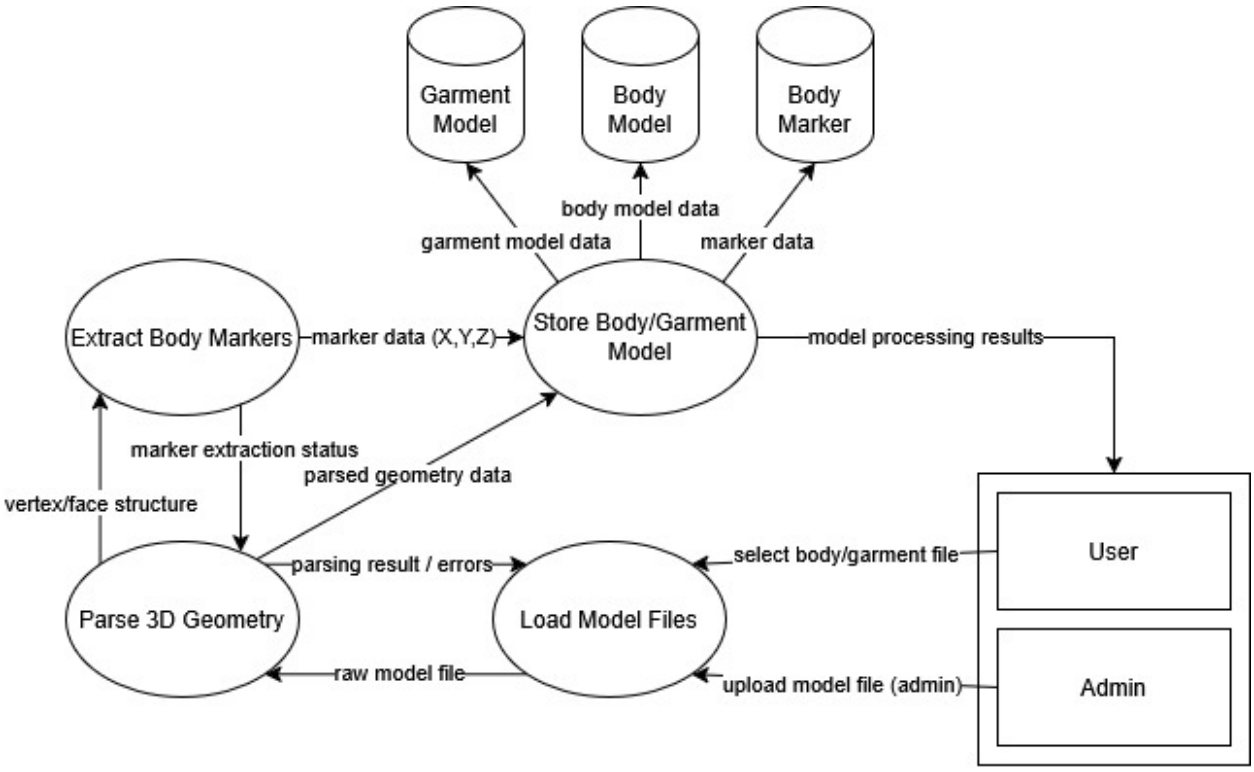


Fig 6. Body & Garment Processing module

BGP.LOAD

As a User, I want to load body and garment model files so that the system can process them and prepare them for analysis.

Actor	**Description**
User	A general system user; uploads and selects 3D model files.
Admin	A privileged user; performs the same actions with elevated system access.

Pre-conditions
1. Dataset has already been uploaded (from previous module). 2. The User or Admin has access to the .obj model files.

Post-conditions
The following conditions must be met when this use case is completed: 1. Files are successfully passed into the module for processing.

2. The system is ready to parse geometric data.

Flow of Events
1. The User navigates to the dataset view page. 2. The User selects a body model file or a garment model file from local storage. 3. The User clicks the “Load Model” button. 4. The file is transferred to the system and forwarded to the parsing process. 5. The system verifies that the file format is supported. 5.1. **Alternative A1**: File format invalid 6. The file is accepted and ready for geometry parsing. 7. The system displays confirmation that the file has been loaded. The use case ends.

Alternative flows:

A1: File format invalid
1. The system displays an error message indicating that the selected file is not supported. 2. The User selects a different file. 2.1. **Alternative A1** repeats if the new file is also invalid. The alternative flow ends, basic flow continues at step 3.

BGP.PARSE

As a User, I want the system to detect vertices and faces so that my model can be used in analysis.

Actor	**Description**
System	System processes files, no user interaction during parsing.

Pre-conditions
1. A valid .obj file has been loaded.

Post-conditions
The following conditions must be met when this use case is completed: 1. The system extracts vertex and face structures. 2. Parsed data is ready for marker extraction and storage.

Flow of Events
1. The module receives a raw .obj file. 2. The system reads all vertex lines. 3. The system reads all face definitions. 4. The system constructs a geometric representation of the model. 5. Any parsing errors are returned to the Load Model Files process. 6. If no errors occurred, the system outputs structured geometry data. 7. The system forwards the result to the Extract Body Markers process. The use case ends.

BGP.MARKERS

As a User, I want body markers to be automatically extracted, so that the system can later compute body-garment distances.

Actor	**Description**
System	System processes files, no user interaction during parsing.

Pre-conditions
1. Geometry has been parsed. 2. The body model contains identifiable marker points.

Post-conditions
The following conditions must be met when this use case is completed: 1. Marker coordinates are extracted. 2. Marker data is stored in the Body Marker data store.

Flow of Events
1. The system scans geometric data for predefined marker points. 2. Marker vertices are matched by naming convention or index. 3. Marker coordinates (X, Y, Z) are extracted. 4. The system validates marker count. 4.1. **Alternative A1**: Marker structure inconsistent 5. Valid markers are saved to the Body Marker data store. 6. System returns success to the User. The use case ends.

Alternative flows:

A1: Invalid marker structure
1. The system displays an error indicating missing or invalid markers. 2. The User must re-upload or correct the model file. The alternative flow ends, basic flow continues at step 1.

BGP.STORE

As a User, I want the system to save all extracted geometry so that the model can be reused later for analysis.

Actor	**Description**
User	A general system user.
Admin	A privileged user with extended management capabilities.

Pre-conditions
1. Parsing and marker extraction succeeded.

Post-conditions
The following conditions must be met when this use case is completed:

1. Model geometry is stored in Body Model or Garment Model data store.
2. The model becomes available for Analysis Session creation.

Flow of Events

1. The system receives parsed geometry.
2. The system identifies whether the file is a body or garment model.
3. The system stores model metadata:
    - Vertex count
    - Face count
    - Model name
4. The system stores geometry into the proper data store.
5. A confirmation message is displayed to the User.
6. The model becomes visible in the dataset details page.

The use case ends.

2.2.5 Analysis Execution Module

The Analysis Execution Module is responsible for running the core processing workflows that compare a user's body model and garment model. This module allows the User and Administrator to start a new analysis session, apply algorithm configuration settings, compute marker-based distances, and generate aggregated comparison results. It ensures that all necessary model data is retrieved, processed, and stored correctly so that later modules (such as the Comparison Reporting Module) can access accurate similarity metrics.

This module serves as the computational core of the system, linking prepared models and markers with algorithmic evaluation techniques.

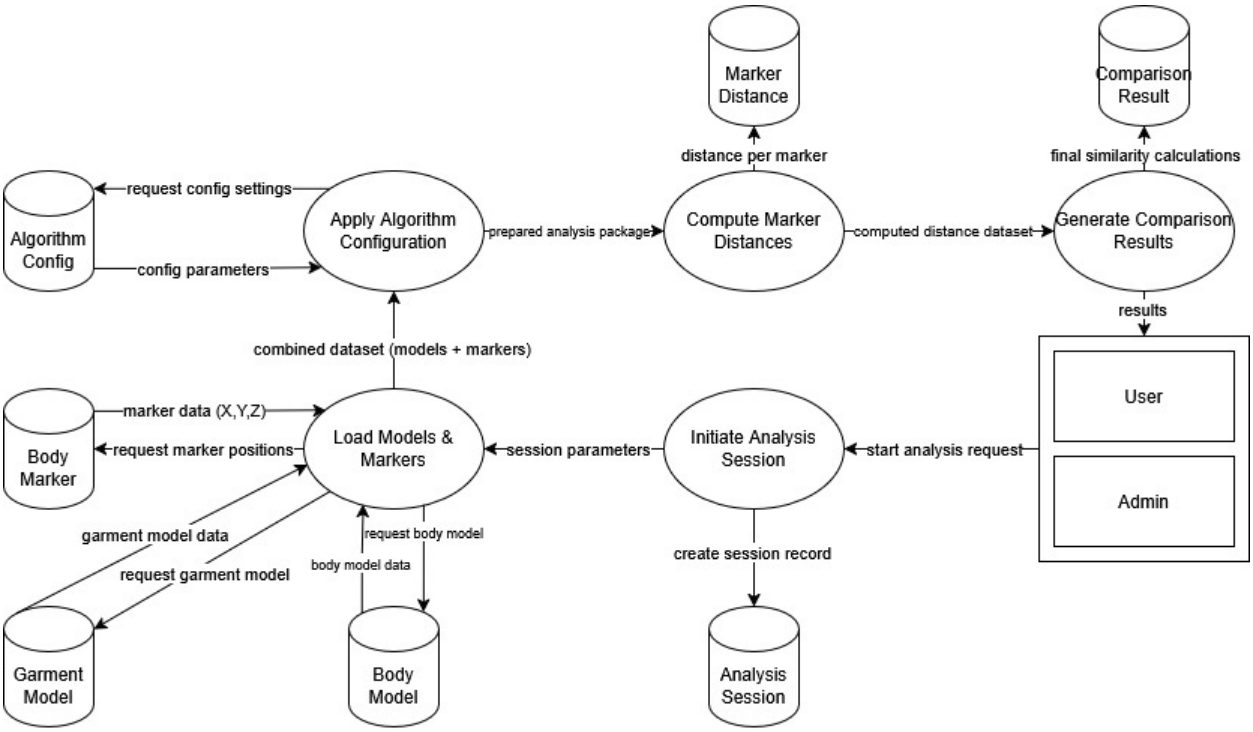


Fig 7. Analysis Execution model

ANALYSIS.START

As a User, I want to initiate a new analysis session so that I can compare my body model with a selected garment model using the algorithm settings of my choice.

Actor	**Description**
User	A general system user.
Admin	A privileged user; has extended control over datasets and analysis configuration.

Pre-conditions
1. User is authenticated. 2. Body Model and Garment Model exist in the system. 3. Dataset has successfully passed preprocessing.

Post-conditions
The following conditions must be met when this use case is completed:

1. A new Analysis Session is created. 2. Required models and markers are loaded for further processing. 3. Algorithm configuration is linked to this session.

Flow of Events
1. The User navigates to the “Start Analysis” page. 2. The User selects a Body Model and a Garment Model. 3. The User optionally selects custom Algorithm Configuration. 4. The User presses the “Start Analysis” button. 4.1. **Alternative A1**: Input is invalid 5. The system creates a new entry in Analysis Session. 6. The system loads the selected models and markers. 7. The User is forwarded to the session overview page. The use case ends.

Alternative flows:

A1: Missing or invalid selection
1. The User presses “Start Analysis” without selecting valid inputs. 2. The system displays a validation error and highlights required fields. 3. User corrects the inputs. The alternative flow ends, basic flow continues at step 4.

ANALYSIS.DISTANCE

As a User, I need the system to compute marker-based distances so that the analysis session can evaluate how the garment fits the body.

Actor	**Description**
System	System computes data, no user interaction during computation.

Pre-conditions

1. A valid analysis session exists.
2. Body model, garment model, and markers are loaded.
3. Algorithm configuration is available.

Post-conditions
The following conditions must be met when this use case is completed: 1. Marker Distance entries are created for each marker. 2. Distances are stored in the data store.

Flow of Events
1. System receives body and garment mesh positions. 2. For each marker, the distance is computed according to selected distance method. 2.1. **Alternative A1**: Marker Error 3. Distance results are saved to Marker Distance. 4. System signals completion to the Comparison Result generator. The use case ends.

Alternative flows:

A1: Missing markers
1. System detects missing or corrupted marker data. 2. A failure message is generated. 3. The session is marked as incomplete. Flow ends.

ANALYSIS.RESULT

As a User, I want to view the similarity results so that I can understand the fit quality of a garment model.

Actor	**Description**
User	A general system user.

Admin	A privileged user; may access any result for troubleshooting or demonstration.

Pre-conditions
1. Marker distances are computed. 2. A valid analysis session exists.

Post-conditions
The following conditions must be met when this use case is completed: 1. A complete Comparison Result record is created. 2. The User receives aggregated similarity metrics.

Flow of Events
1. The System aggregates all marker distances. 2. The System computes: • Average difference • Max difference • Min difference • Similarity percentage • Volume similarity • Combined similarity 3. The System creates a new Comparison Result entry. 4. The User is shown the results page. The use case ends.

2.2.6 Distance & Fit Calculation Module

The Distance & Fit Calculation Module is responsible for computing precise geometric measurements between the user's body model and the selected garment model. After an analysis session has been initiated and the necessary models, markers, and algorithm configuration have

been prepared, this module performs detailed per-marker distance evaluation and calculates garment fit metrics. It takes the prepared analysis dataset, applies the appropriate distance method, and stores computed distances in the system. The module ensures that all markers are processed consistently and makes the computed distance data available for subsequent similarity aggregation in the Comparison Reporting Module.

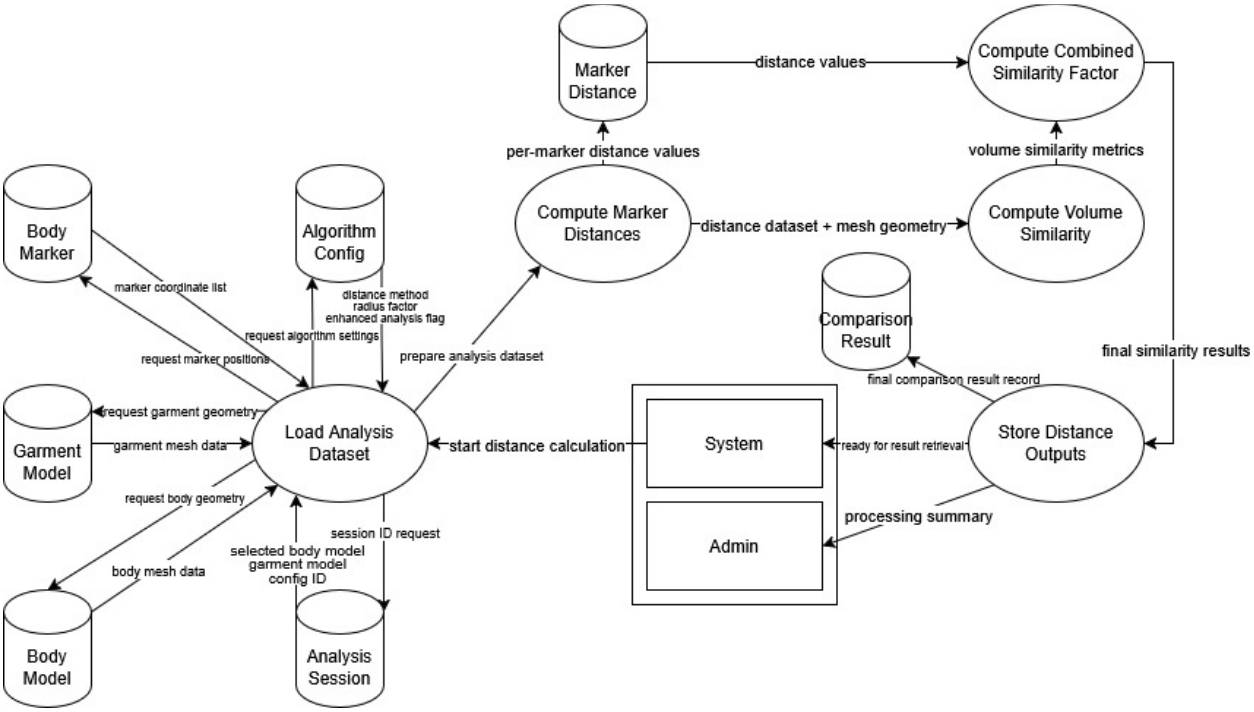


Fig 8. Distance & Fit Calculation module

DISTANCE.CALCULATE

As a User, I need the system to calculate per-marker distances so that it can evaluate the geometric difference between the body and garment models.

Actor	**Description**
System	System computes data, no user interaction during computation.

Pre-conditions
1. Analysis Session is initialized. 2. Models, markers, and configuration are stored in the database. 3. The analysis dataset loads successfully.

Post-conditions
The following conditions must be met when this use case is completed: 1. All marker distances are computed. 2. Distance values are saved in Marker Distance data store.

Flow of Events
1. System identifies all marker coordinates. 1.1. **Alternative A1**: Marker Error 2. System loads garment mesh surface. 3. For each marker, the system computes distance using the configured method. 4. System writes each distance entry into Marker Distance. 5. System signals completion to next processing step. The use case ends.

Alternative flows:

A1: Missing markers
1. System detects missing or corrupted marker data. 2. System logs an error and stops processing. 3. Analysis Session is flagged as failed. Use case terminates.

DISTANCE.VOLUME

As a User, I need the system to compute volume-based similarity so that the fit analysis reflects global geometric similarity, not only per-marker distances.

Actor	**Description**
System	System computes data, no user interaction during computation.

Pre-conditions
1. Marker distances are computed.

2. Body and garment 3D meshes are available.

Post-conditions
The following conditions must be met when this use case is completed: 1. Volume similarity metric is computed. 2. Metric is passed to next stage.

Flow of Events
1. System loads body and garment meshes. 1.1. **Alternative A1**: Mesh Error 2. System applies the configured volume approximation algorithm. 3. System computes normalized overlap/volume ratio. 4. Volume similarity is forwarded to combined metric calculation. The use case ends.

Alternative flows:

A1: Volume computation fails
1. Mesh is incomplete or corrupted. 2. Volume similarity is set to "N/A." 3. The system continues with distance-based similarity only.

DISTANCE.COMBINED

As a User, I need the system to generate a combined similarity score so that the user receives a single, interpretable metric of garment fit.

Actor	**Description**
System	System computes data, no user interaction during computation.

Pre-conditions
1. Marker distances are computed.

2. Volume similarity is computed (when available).

Post-conditions
The following conditions must be met when this use case is completed: 1. A Comparison Result record is created. 2. Final similarity values are stored.

Flow of Events
1. System collects: • average difference • maximum difference • minimum difference • similarity percentage • volume similarity 1.1. **Alternative A1**: Missing Metrics 2. System computes the combined similarity using algorithm configuration. 3. System creates a new Comparison Result entry. 4. The result becomes available to User and Administrator. The use case ends.

Alternative flows:

A1: Missing metrics
1. One or more distance or volume values are missing. 2. System computes similarity using available metrics only. 3. Result is stored with missing-value indicators.

2.2.7 Comparison Reporting Module

The Comparison Reporting Module is responsible for retrieving, formatting, and presenting the results of a completed analysis session to the User or Administrator. After distance and similarity metrics are computed and stored, this module provides access to the results, generates structured reports, and allows the user to export or download the outcome for reference or further analysis. The module ensures that final comparison results are accessible, readable, and exportable, giving users a clear understanding of garment fit quality based on body-to-garment comparison.
It serves as the endpoint of the analytical pipeline, transforming computed data into meaningful reports.

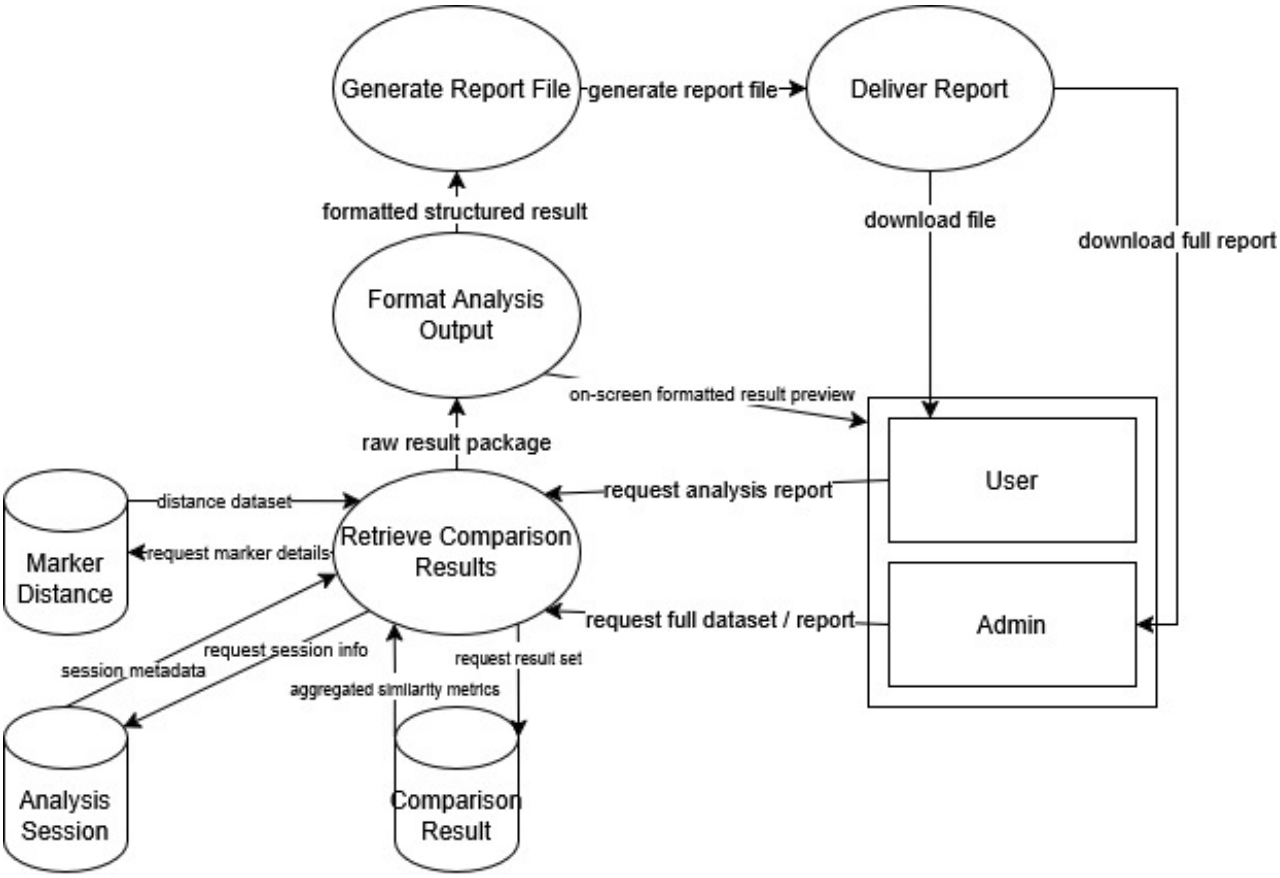


Fig 9. Comparison Reporting module

REPORT.VIEW

As a User, I want to view the results of my analysis so that I can understand the fit of the garment model.

Actor	Description
User	A general system user who performed an analysis session.

Pre-conditions
1. The User is authenticated. 2. A Comparison Result exists for the selected analysis session. 3. Distance & Volume calculations are complete.

Post-conditions
The following conditions must be met when this use case is completed: 1. The User has viewed the formatted comparison report. 2. The User can choose to download or export the result.

Flow of Events
1. User navigates to “Analysis Results.” 2. User selects an existing Analysis Session. 3. System retrieves Comparison Result. 3.1. **Alternative A1**: No Comparison Result 4. System fetches marker distances (optional). 5. System formats the metrics (tables, percentages, summaries). 6. System displays the formatted report on-screen. 7. The User reviews the results. The use case ends.

Alternative flows:

A1: No result available
1. System detects missing result entry. 2. A message is displayed: “No completed analysis available.” 3. User returns to the Analysis Dashboard.

REPORT.DOWNLOAD

As a User, I want to download my analysis report so that I can store it, share it, or use it for offline review.

Actor	Description
User	A general system user.

Pre-conditions
1. A formatted report exists. 2. All file generation components are operational.

Post-conditions
The following conditions must be met when this use case is completed: 1. A report file is generated. 2. User receives a binary file download.

Flow of Events
1. User clicks “Download Report.” 2. System formats the result data (if not already cached). 3. System generates the report file (PDF, JSON, or text). 3.1. **Alternative A1**: File Generation Failure 4. System presents a download link. 5. User downloads the file. The use case ends.

Alternative flows:

A1: File generation failure
1. System fails to generate the file. 2. Error message appears: “Unable to generate report.”

User may retry (basic flow step 1) or contact administrator.

REPORT.ADMIN.VIEWALL

As an Administrator, I want to view all comparison results so that I can verify analysis outcomes, troubleshoot issues, or demonstrate system behavior.

Actor	**Description**
Admin	A privileged user; has full access to all results and dataset details.

Pre-conditions
1. Administrator is authenticated. 2. At least one completed Analysis Session exists.

Post-conditions
The following conditions must be met when this use case is completed: 1. Administrator views all results for the selected session. 2. Administrator may download detailed reports.

Flow of Events
1. Administrator navigates to the “Results Overview.” 1.1. **Alternative A1**: No Sessions 2. Administrator selects any past analysis session. 3. System retrieves: • Comparison Result • Marker Distance dataset • Session metadata 4. System formats detailed extended report. 5. Administrator views full structured dataset. 6. Administrator may download a full version of the report. The use case ends.

Alternative flows:

A1: No sessions available
1. System displays: “No completed sessions available.” 2. Admin returns to dashboard.

3. SOFTWARE DESIGN DESCRIPTION

3.1 Database Design

The Logical Model Diagram was created based on the functional requirements and Conceptual database model **(Crowfoot syntax)**. The Physical Model Diagram was created based on the logical database model.

The data types, terms and parameters used may vary depending on the DBMS. For this system, the terms for Microsoft SQL will be used.

3.1.1 Logical Model

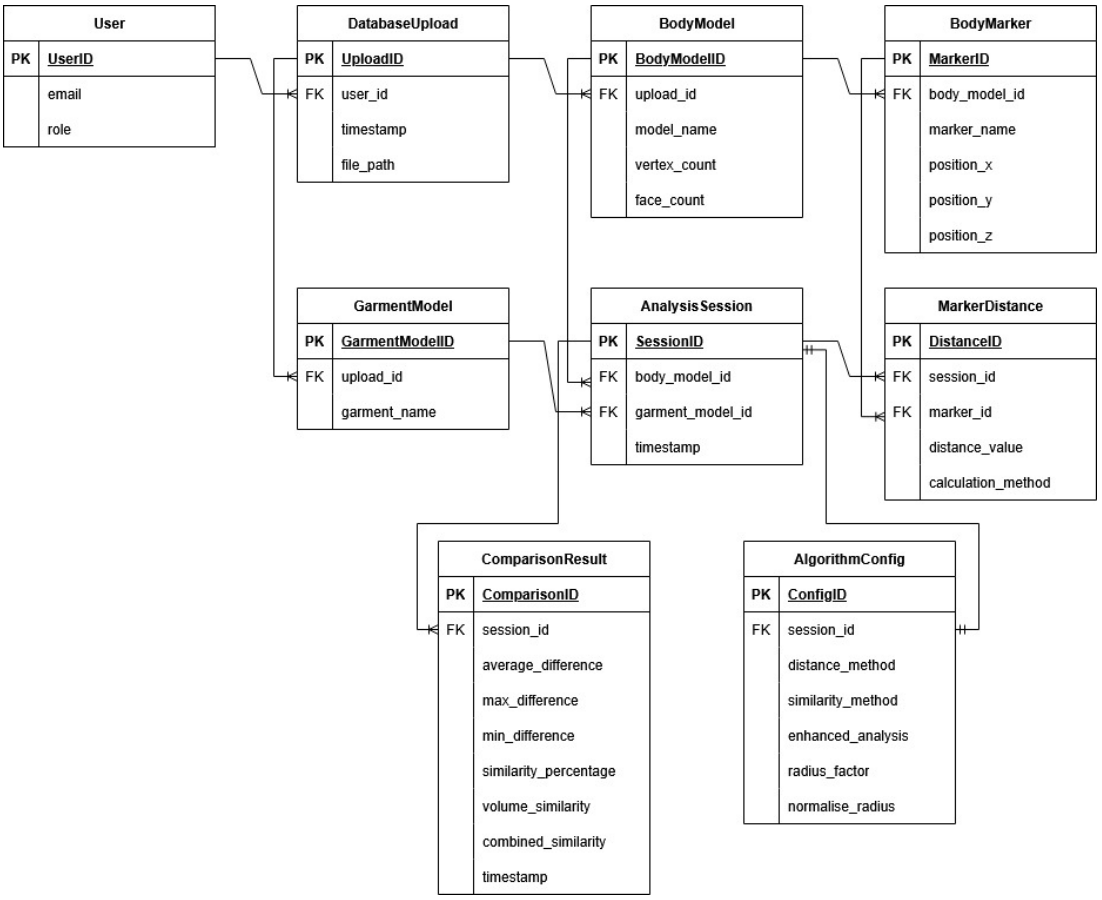


Fig 10. Logical Model Diagram

3.1.2 Physical Model

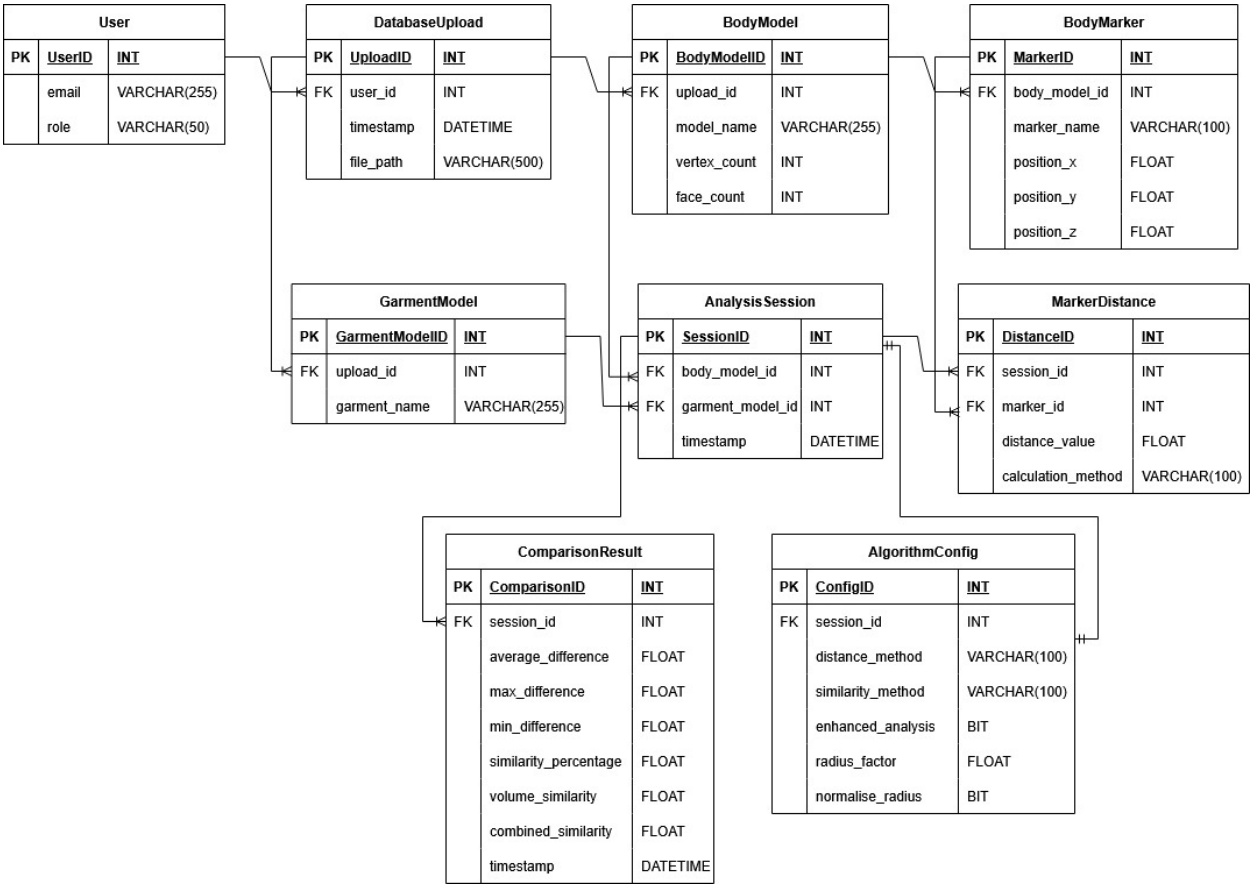


Fig 11. Physical Model Diagram

3.1.3 Database Entity description

User Entity

The User entity stores essential information about system users. It represents individuals who interact with the system, including regular users and administrators. The entity is responsible for authentication, authorization, and ownership of uploaded datasets.

Entity references: USER.DATASET.UPLOAD, USER.DATASET.VIEW

Key	Column name	Data type	Nullable	Other parameters	Description
PK	user_id	INT	No	IDENTITY, PRIMARY KEY	Unique user identifier
	email	VARCHAR(255)	No	UNIQUE	User email address
	role	VARCHAR(50)	No		User role (e.g., user, admin)

Table 2. User entity description

Dataset Upload Entity

The Dataset Upload entity represents uploaded datasets that serve as containers for body and garment models. Each upload is linked to a specific user and stores metadata related to the upload event.

Entity references: USER.DATASET.UPLOAD, USER.DATASET.VIEW, USER.DATASET.DELETE

Key	Column name	Data type	Nullable	Other parameters	Description
PK	upload_id	INT	No	IDENTITY, PRIMARY KEY	Upload identifier
FK	user_id	INT	No	REFERENCES User(user_id)	Upload owner
	timestamp	DATETIME	No	DEFAULT GETDATE()	Upload time

	file_path	VARCHAR(500)	No		Path to uploaded dataset

Table 3. Dataset Upload entity description

Body Model Entity

The Body Model entity stores metadata about 3D body meshes uploaded into the system. These models are used as geometric references during fitting and distance analysis.

Entity references: BGP.LOAD, BGP.PARSE, BGP.STORE, ANALYSIS.START

Key	Column name	Data type	Nullable	Other parameters	Description
PK	body_model_id	INT	No	IDENTITY, PRIMARY KEY	Body model identifier
FK	upload_id	INT	No	REFERENCES DatasetUpload(upload_id)	Related dataset
	model_name	VARCHAR(255)	No		Body model name
	vertex_count	INT	No		Number of vertices
	face_count	INT	No		Number of faces

Table 4. Body Model entity description

Body Marker Entity

The Body Marker entity defines anatomical markers associated with a body model. These markers provide reference points for distance calculations and comparative analysis.

Entity references: BGP.MARKERS, DISTANCE.CALCULATE

Key	Column name	Data type	Nullable	Other parameters	Description
PK	marker_id	INT	No	IDENTITY, PRIMARY KEY	Marker identifier

FK	body_model_id	INT	No	REFERENCES BodyModel(body_model_id)	Associated body model
	marker_name	VARCHAR(255)	No		Marker label
	position_x	FLOAT	No		X coordinate
	position_y	FLOAT	No		Y coordinate
	position_z	FLOAT	No		Z coordinate

Table 5. Body Marker entity description

Garment Model Entity

The Garment Model entity stores metadata for garment meshes that are analysed against body models during fitting simulations.

Entity references: BGP.LOAD, BGP.PARSE, BGP.STORE, ANALYSIS.START

Key	Column name	Data type	Nullable	Other parameters	Description
PK	garment_model_id	INT	No	IDENTITY, PRIMARY KEY	Garment model identifier
FK	upload_id	INT	No	REFERENCES DatasetUpload(upload_id)	Related dataset
	garment_name	VARCHAR(255)	No		Garment name

Table 6. Garment Model entity description

Analysis Session Entity

The Analysis Session entity represents a single execution of a fitting analysis between a body model and a garment model. It serves as a central entity linking configuration, distance calculations, and results.

Entity references: ANALYSIS.START, ANALYSIS.DISTANCE, ANALYSIS.RESULT

Key	Column name	Data type	Nullable	Other parameters	Description
PK	session_id	INT	No	IDENTITY, PRIMARY KEY	Analysis session identifier
FK	body_model_id	INT	No	REFERENCES BodyModel(body_model_id)	Body model used
FK	garment_model_id	INT	No	REFERENCES GarmentModel(garment_model_id)	Garment model used
	timestamp	DATETIME	No	DEFAULT GETDATE()	Execution time

Table 7. Analysis Session entity description

Marker Distance Entity

The Marker Distance entity stores calculated distances between body markers and the garment surface. Each entry corresponds to a specific marker within an analysis session.

Entity references: DISTANCE.CALCULATE, ANALYSIS.DISTANCE

Key	Column name	Data type	Nullable	Other parameters	Description
PK	distance_id	INT	No	IDENTITY, PRIMARY KEY	Distance identifier
FK	session_id	INT	No	REFERENCES AnalysisSession(session_id)	Analysis session
FK	marker_id	INT	No	REFERENCES BodyMarker(marker_id)	Body marker

	distance_value	FLOAT	No		Calculated distance
	calculation_method	VARCHAR(100)	No		Method used

Table 8. Marker Distance entity description

Comparison Result Entity

The Comparison Result entity stores aggregated similarity and distance metrics derived from an analysis session. These values are used for reporting and comparison across different fitting scenarios.

Entity references: ANALYSIS.RESULT, REPORT.VIEW, REPORT.DOWNLOAD

Key	**Column name**	**Data type**	**Nullable**	**Other parameters**	**Description**
PK	comparison_id	INT	No	IDENTITY, PRIMARY KEY	Result identifier
FK	session_id	INT	No	REFERENCES AnalysisSession (session_id)	Related session
	average_difference	FLOAT	No		Mean distance
	max_difference	FLOAT	No		Maximum distance
	min_difference	FLOAT	No		Minimum distance
	similarity_percentage	FLOAT	No		Similarity score
	volume_similarity	FLOAT	No		Volume-based similarity
	combined_similarity	FLOAT	No		Combined metric

	timestamp	DATETIME	No	DEFAULT GETDATE()	Result timestamp

Table 9. Comparison Result entity description

Algorithm Configuration Entity

The Algorithm Configuration entity stores configuration parameters used during an analysis session. It ensures reproducibility and traceability of analysis results.

Entity references: ANALYSIS.START, ANALYSIS.DISTANCE

Key	Column name	Data type	Nullable	Other parameters	Description
PK	config_id	INT	No	IDENTITY, PRIMARY KEY	Configuration identifier
FK	session_id	INT	No	REFERENCES AnalysisSession (session_id)	Analysis session
	distance_method	VARCHAR(100)	No		Distance algorithm
	similarity_method	VARCHAR(100)	No		Similarity metric
	enhanced_analysis	BIT	No		Enhanced analysis flag
	radius_factor	FLOAT	Yes		Radius scaling
	normalize_radius	BIT	No		Normalisation flag

Table 10. Algorithm Configuration entity description

3.2 Design of User Interface

3.2.1 Login window

The login window serves as the entry point to the application. It allows users to authenticate themselves using an email address. Only email addresses that are registered in the system database are permitted to access the main application.

Unauthenticated users are restricted to the login window and cannot access any analysis functionality. Upon successful authentication, the user is redirected to the main analysis page.

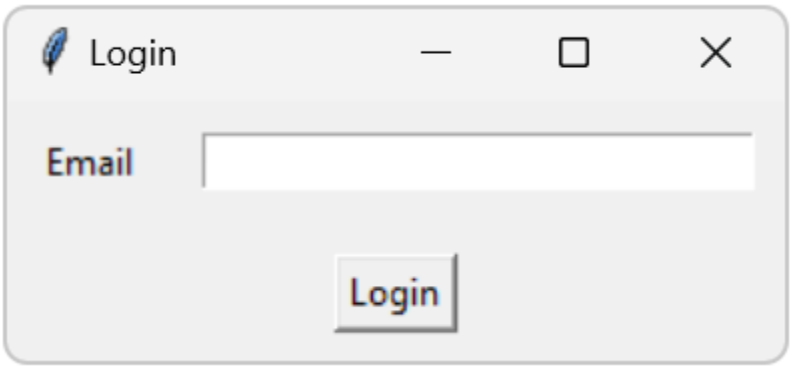


Fig 12. Login window

3.2.2 Main analysis window

The main analysis page provides access to the core functionality of the system. This page allows authenticated users to configure and execute garment fitting analysis sessions.

A file menu is located in the top-left corner of the interface and is visible on all pages. The file menu provides the following options:

- Open analysis configuration
- Save analysis configuration
- Load a previously executed analysis session

Within the main page, users can select body and garment model files for analysis, configure algorithm parameters such as distance calculation methods and similarity metrics, and initiate the analysis process (ANALYSIS.START). After execution, the analysis results are displayed within the interface, including numerical metrics such as average difference and similarity values.

The computed results are shown at the bottom of the window, allowing users to immediately review the outcome of the analysis. Additionally, users may visualize the results and export them for further use (REPORT.VIEW, REPORT.DOWNLOAD).

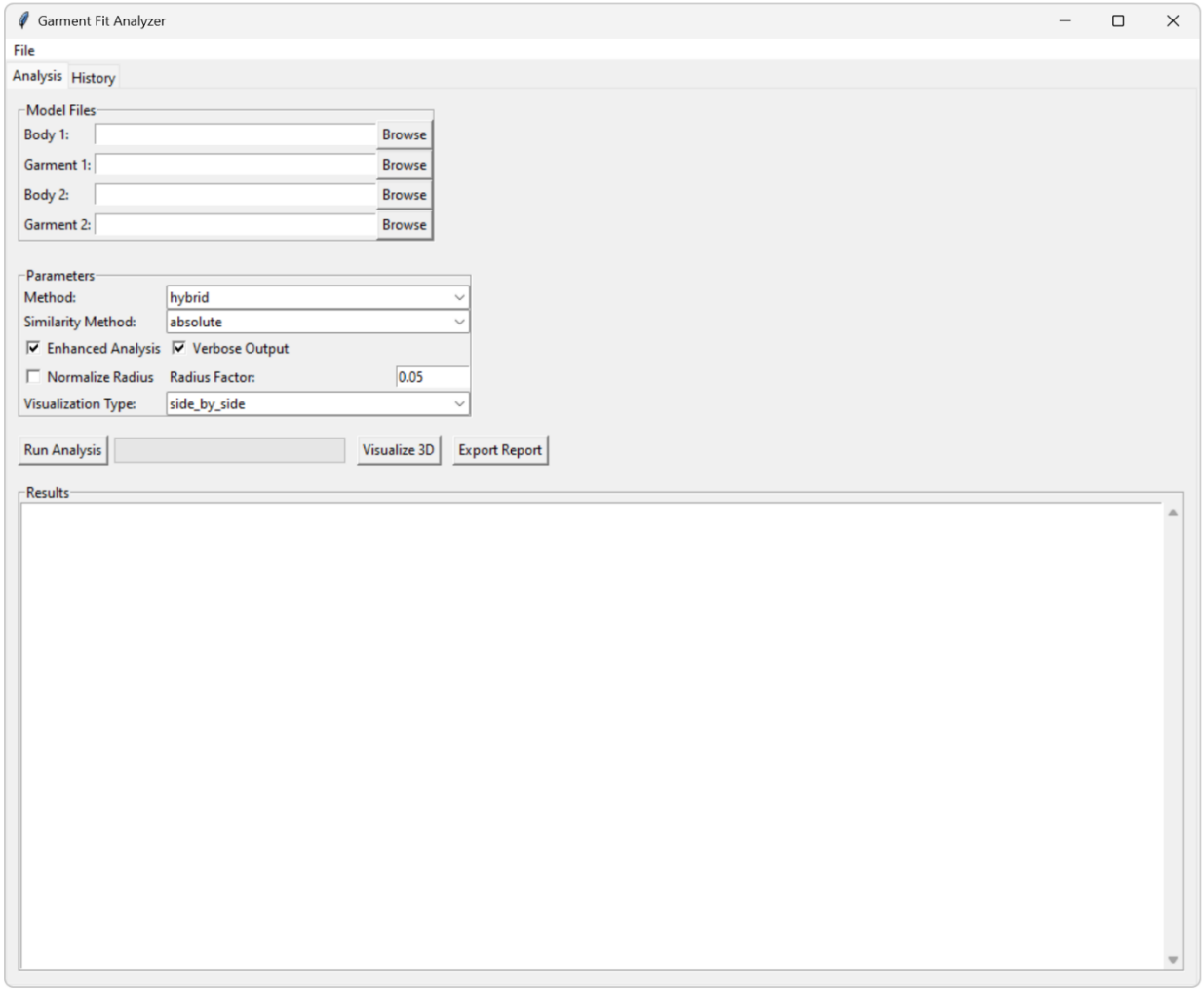


Fig 13. Main window

3.2.3 Visualization window

The visualization view displays graphical representations of garment fitting results, such as color-coded heatmaps and comparative visualizations mapped onto the garment geometry. This view is used solely for result interpretation and does not provide additional user interaction beyond visual inspection.



Fig 14. Side by side visualization

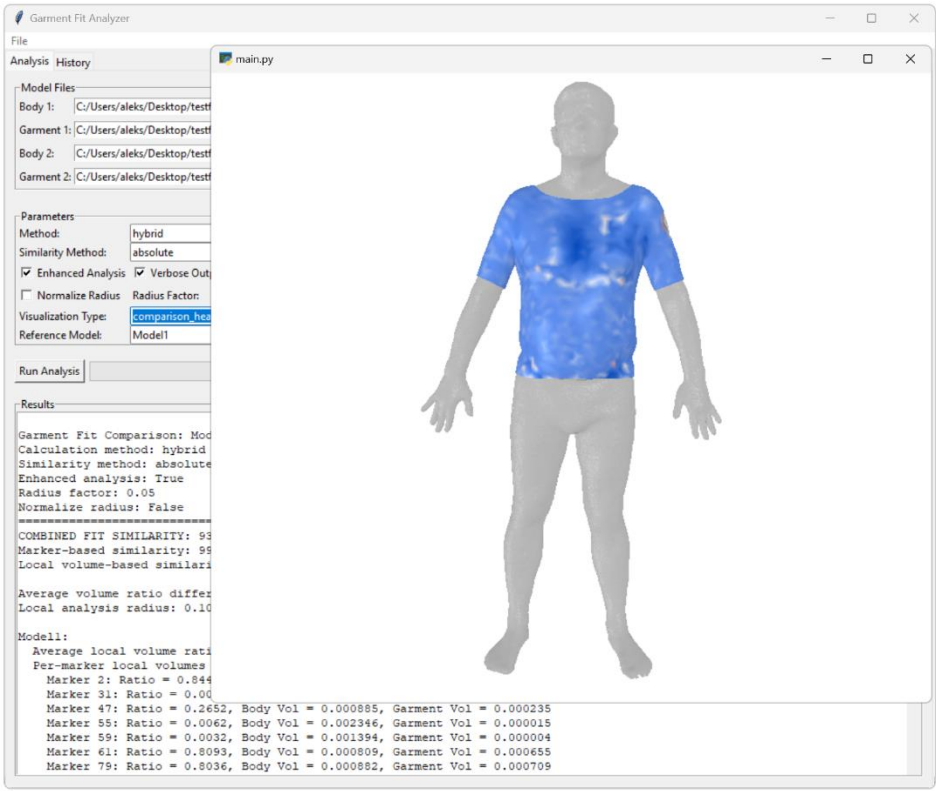


Fig 15. Heatmap visualization

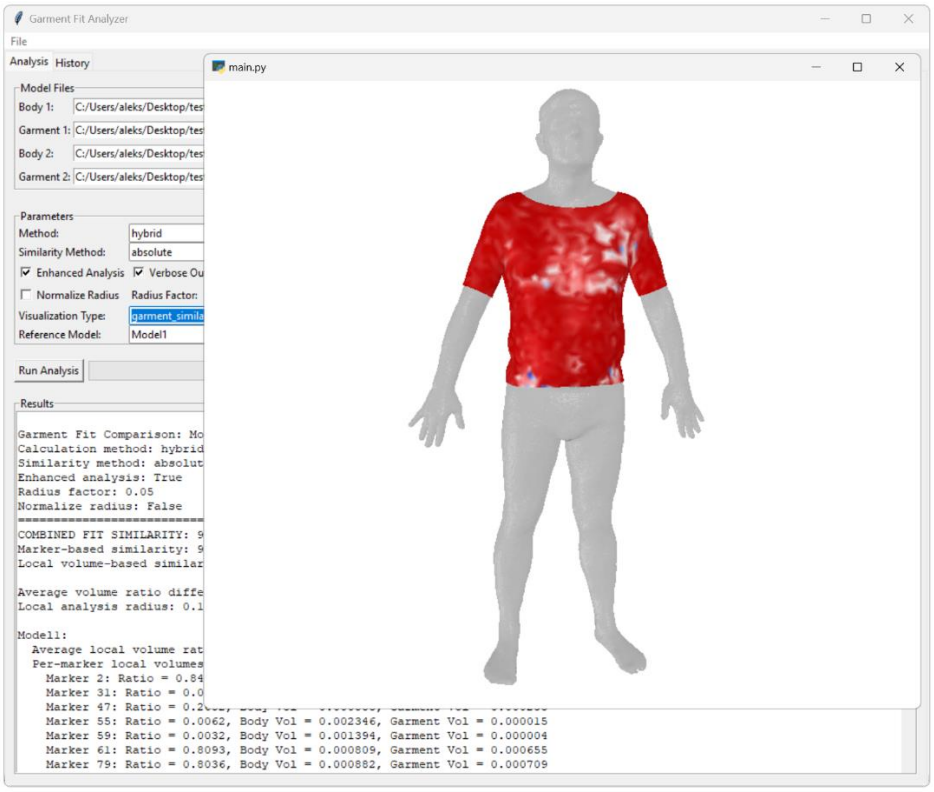


Fig 16. Garment similarity visualization

3.2.4 Analysis history window

The analysis history page allows users to review previously executed analysis sessions. The page displays a list of analysis sessions including the session identifier, execution date, input files used for analysis, average distance difference, and similarity percentage.

At the bottom of the page, the following actions are available:

- Refresh the history list
- Load the selected analysis session for re-evaluation
- View detailed results of the selected analysis session

Users with administrative privileges have access to an additional function that allows deletion of selected analysis sessions from the history.

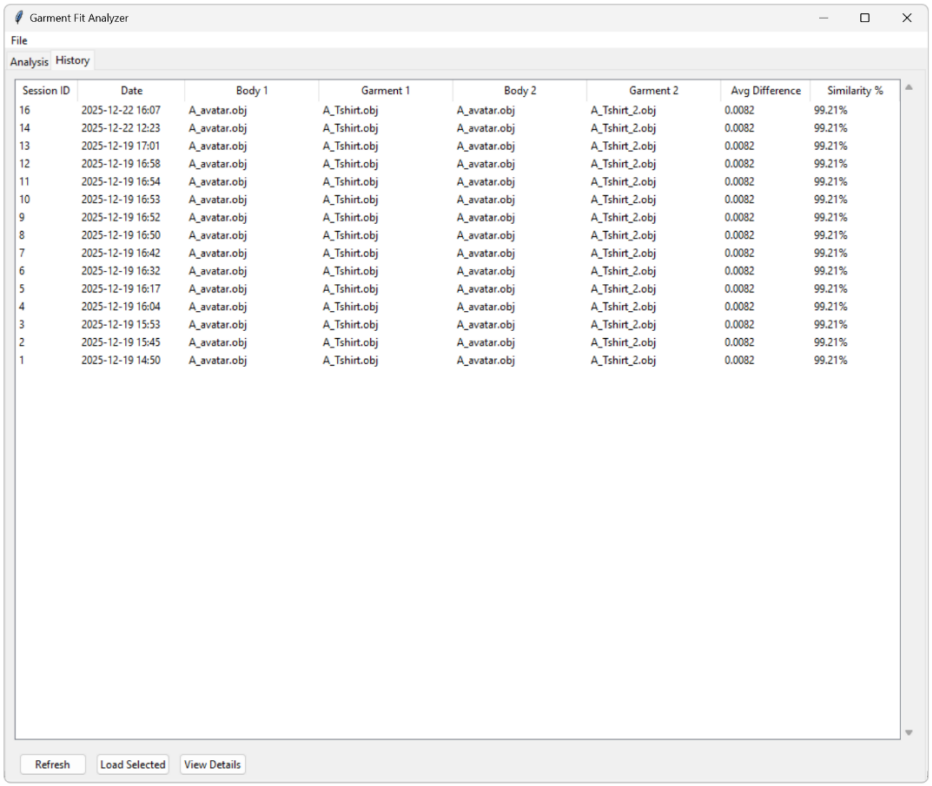


Fig 17. History window

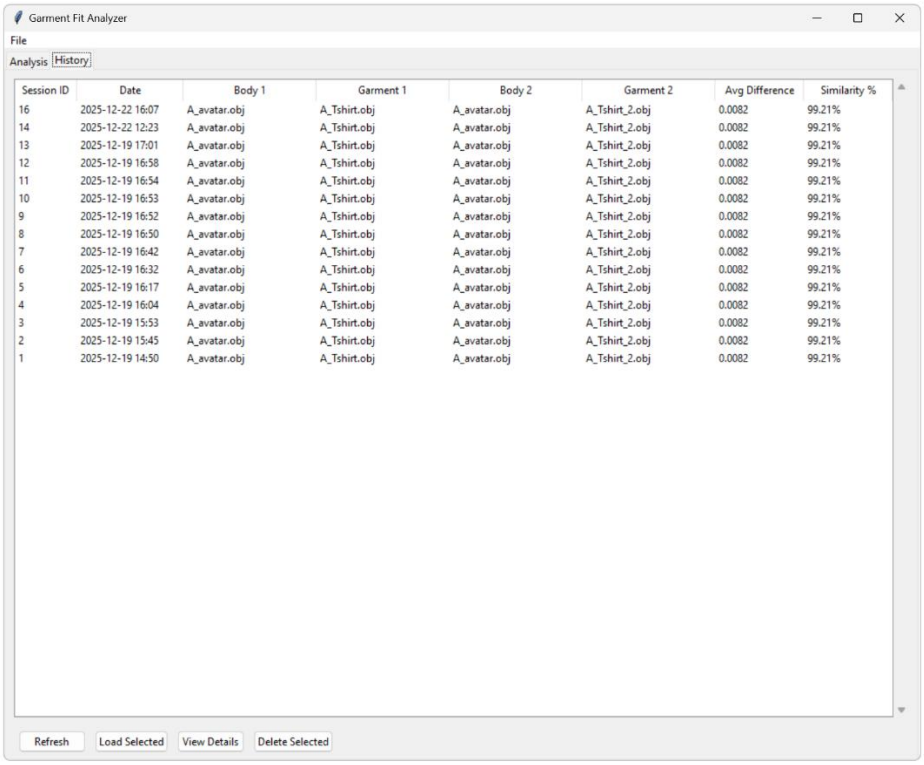


Fig 18. History window (admin view)

3.3 Functional Design

3.3.1 Class Diagram

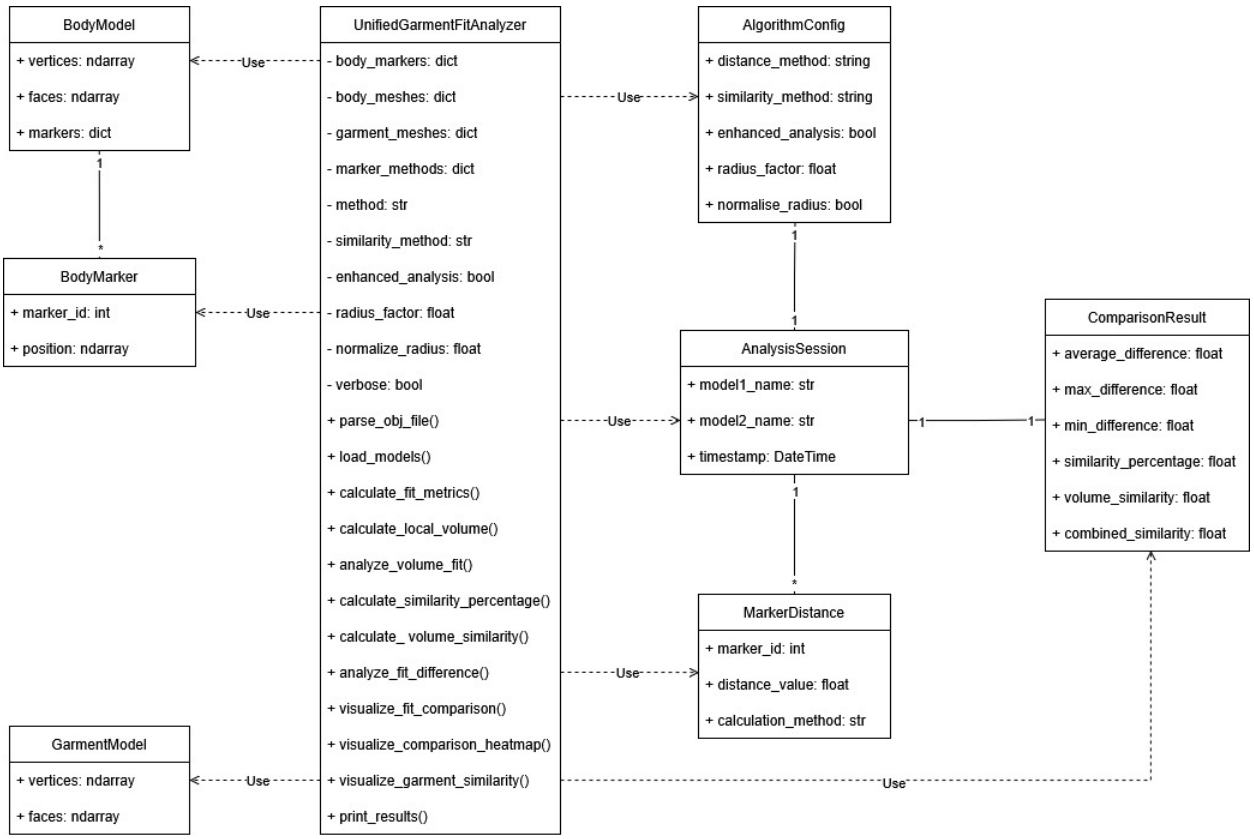


Fig 19. System class diagram

3.3.2 Algorithm flowcharts

3.3.2.1 User Authentication algorithm

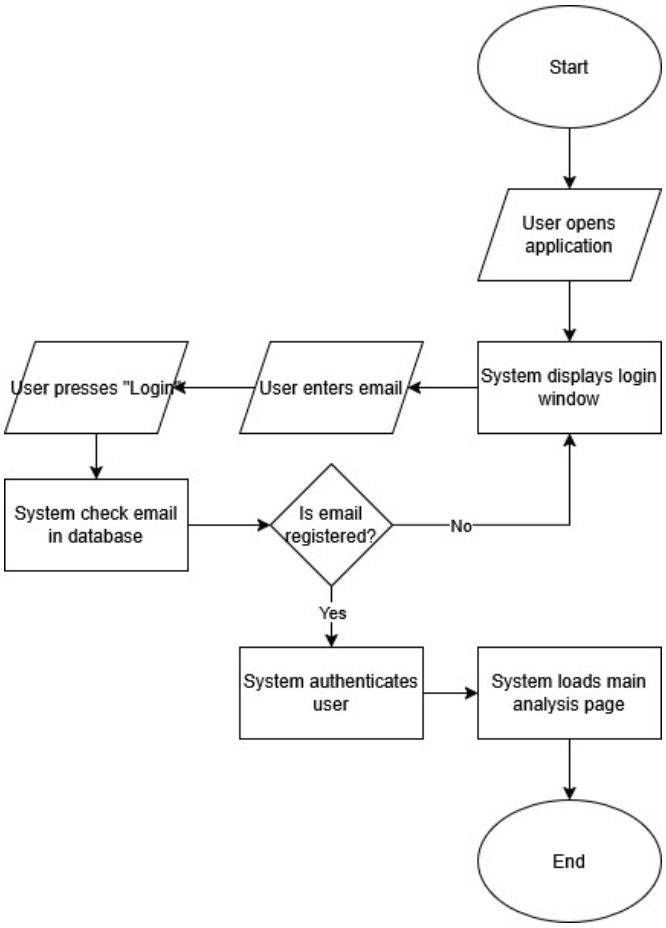


Fig 20. User Authentication algorithm

3.3.2.2 Analysis Configuration algorithm

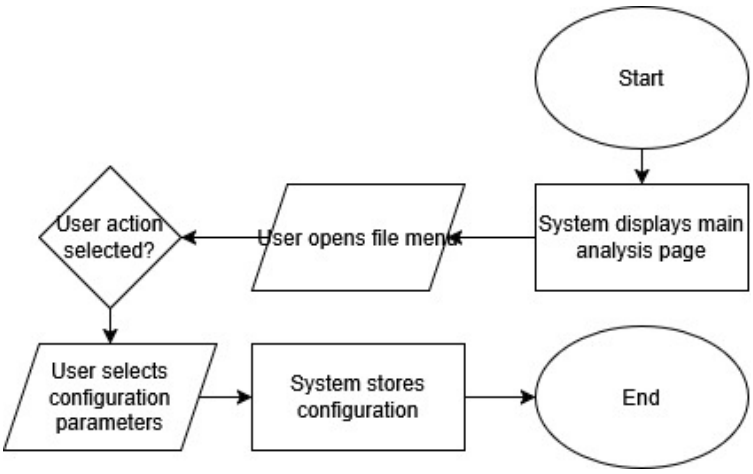


Fig 21. Analysis Configuration algorithm

3.3.2.3 Model Loading algorithm

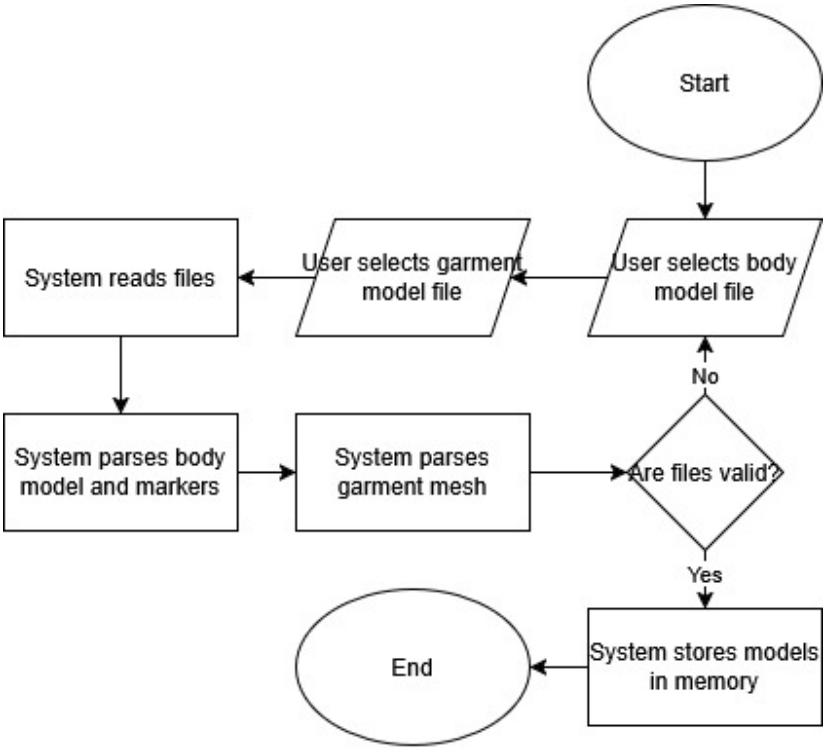


Fig 22. Model Loading algorithm

3.3.2.3 Fit Distance Calculation algorithm

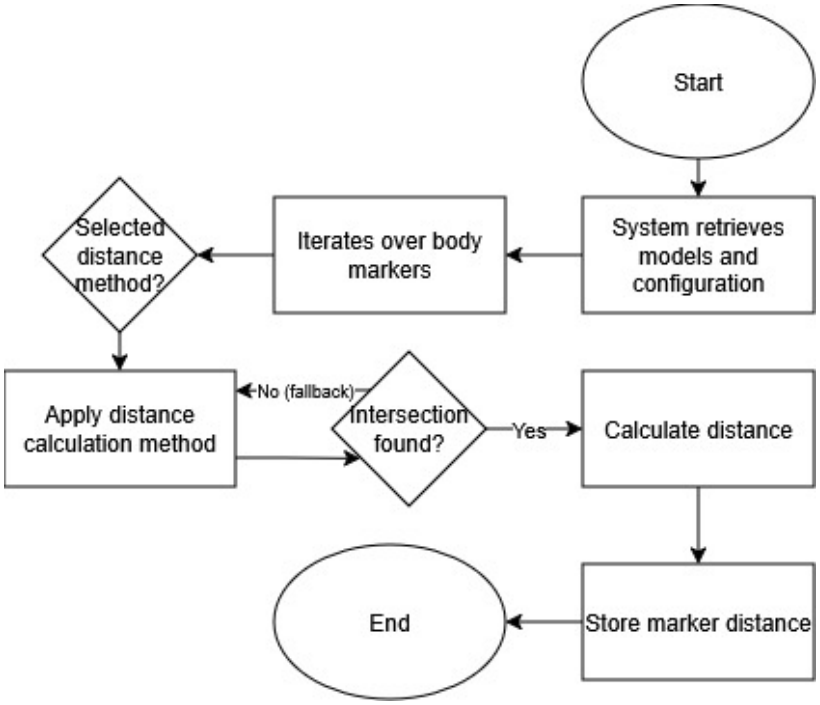


Fig 23. Fit Distance Calculation algorithm

3.3.2.4 Enhanced Volume Analysis algorithm

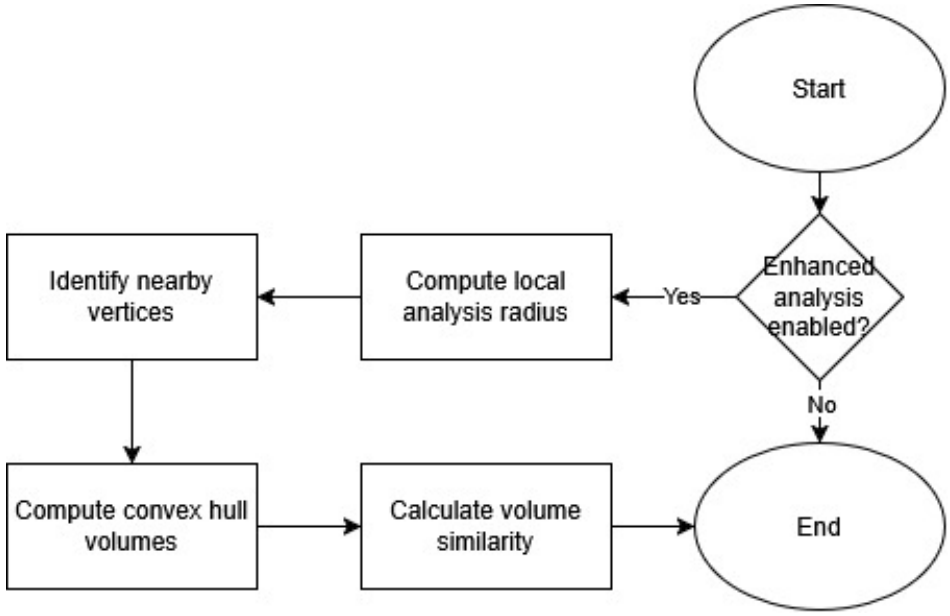


Fig 24. Enhanced Volume Analysis algorithm

3.3.2.5 Fit Comparison algorithm

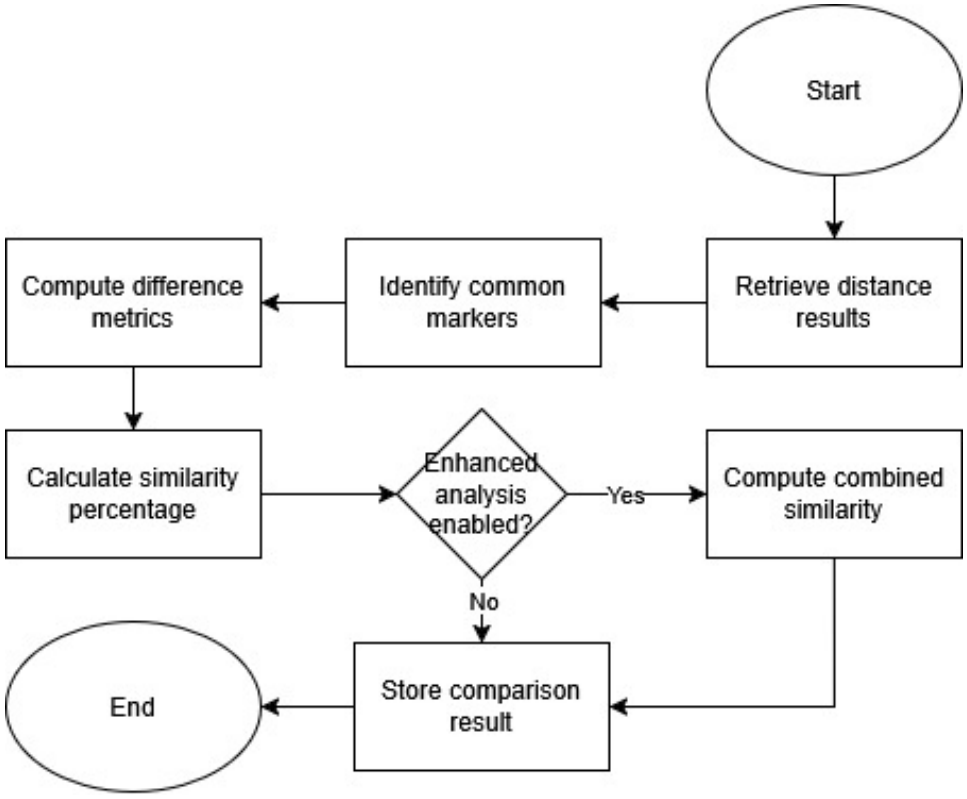


Fig 25. Fit Comparison algorithm

3.3.2.6 Result Visualization algorithm

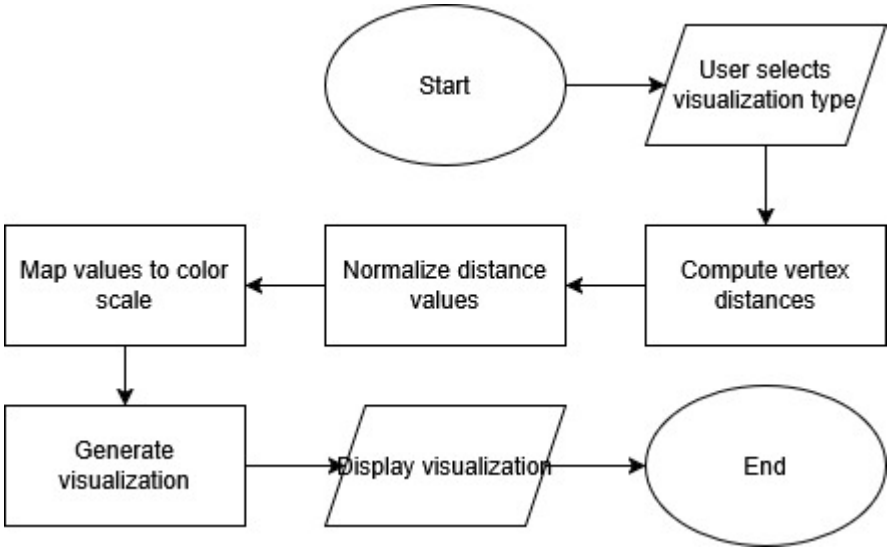


Fig 26. Result Visualization algorithm

3.3.2.7 Analysis History Management algorithm

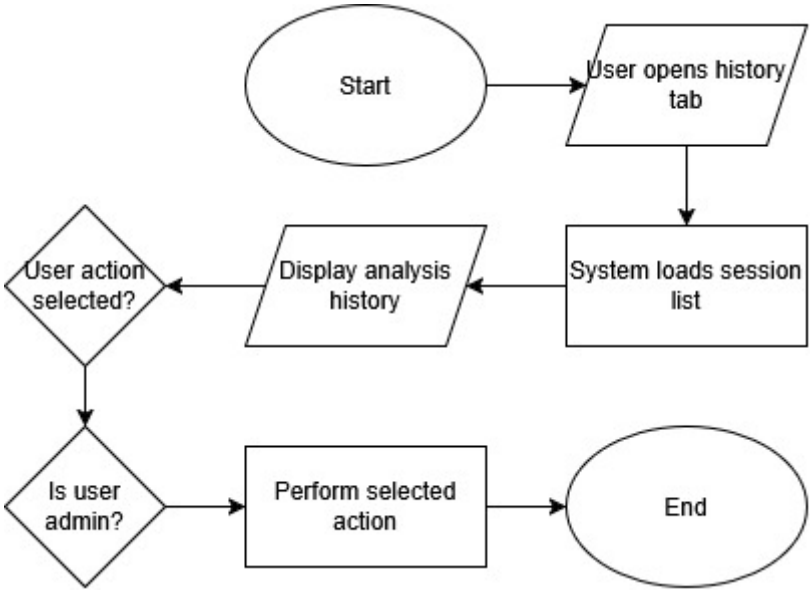


Fig 27. Analysis History Management algorithm

4. TESTING

4.1 Testing Plan

4.1.1 Scope and objectives

The testing activities for this project focus on validating the correctness and reliability of the core computational logic implemented in the system. Testing primarily covers algorithmic functionality related to dataset processing, distance calculations, fit analysis, and comparison result generation. Emphasis is placed on verifying the correctness of numerical outputs, intermediate calculations, and data transformations performed by the system.

The scope of testing does not include graphical user interface layout, visual styling, or subjective user experience aspects. While the application includes visualization components, these are tested only at the level of data correctness (i.e., whether the correct data is passed for visualization), rather than visual appearance or usability. Informal user feedback may still be used to identify usability issues, but such evaluation is outside the formal testing scope.

The main objectives of testing are:

- To ensure that all analytical algorithms execute correctly and consistently
- To verify the accuracy of calculated distances, similarity metrics, and comparison results
- To confirm that analysis sessions are correctly created, stored, loaded, and reproduced
- To ensure that invalid or incomplete input data is handled gracefully without system failure
- To validate that exported and reported results match internally computed values

4.1.2 Selected Framework

The testing framework selected for this project is pytest, a widely used testing framework for Python applications. Pytest provides a simple and expressive syntax for writing unit tests, supports parameterized testing, and integrates well with numerical and scientific Python libraries commonly used for geometric and mathematical computations.

In addition to pytest, Python's built-in unittest utilities and assertion mechanisms are used where appropriate for validating edge cases and exception handling. This testing setup allows efficient

verification of individual functions, algorithm components, and end-to-end analysis workflows without requiring a running graphical interface or external services.

4.2 Test Journal

4.2.1 Test statement coverage and decision coverage

Test Name	Tested algorithm	Number of executed statements	Statement coverage	Number of decision paths	Decision coverage
LoginSuccessful	User Authentication	6/6	100%	2/2	100%
LoginUnsuccessful	User Authentication	6/6	100%	2/2	100%
AdminPrivilegesAssigned	User Authentication	6/6	100%	3/3	100%
DatasetUploadValid	Model Loading	7/8	88%	3/4	75%
DatasetUploadInvalid	Model Loading	7/8	88%	3/4	75%
BodyAndGarmentLoaded	Model Loading	8/8	100%	4/4	100%
MissingMarkersHandled	Model Loading	6/8	75%	3/4	75%
DefaultConfigurationUsed	Analysis Configuration	5/6	83%	2/3	67%
CustomConfigurationApplied	Analysis Configuration	6/6	100%	3/3	100%
DistanceCalculationValid	Fit Distance Calculation	8/9	89%	4/5	80%
DistanceMethodVariation	Fit Distance Calculation	8/9	89%	5/5	100%

EnhancedVolumeEnabled	Enhanced Volume Analysis	6/7	86%	3/4	75%
VolumeSimilarityComputed	Enhanced Volume Analysis	7/7	100%	4/4	100%
FitMetricsComputed	Fit Comparison	8/8	100%	4/4	100%
SimilarityRangeValidation	Fit Comparison	7/8	88%	3/4	75%
HeatmapGenerated	Result Visualization	6/6	100%	3/3	100%
ReportExported	Result Export	6/6	100%	3/3	100%
HistoryListed	Analysis History Management	5/6	83%	2/3	67%
HistoryReload	Analysis History Management	6/6	100%	3/3	100%
AdminDeleteAllowed	Analysis History Management	6/6	100%	3/3	100%
NonAdminDeleteDenied	Analysis History Management	6/6	100%	3/3	100%

Table 11. Statement and decision coverage of tests

4.2.2 Test scope and test levels

The testing strategy for the system followed a structured, multi-level approach focusing on correctness, robustness, and reproducibility of the implemented algorithms. Testing was

performed manually and programmatically by the developer and project manager throughout the development process.

Three conceptual test levels were applied:

- Unit-level testing focused on individual computational components, such as distance calculations, marker parsing logic, similarity metric computation, and volume estimation functions. These tests verified that each function produces expected outputs for controlled input data, including boundary cases and invalid inputs.
- Integration testing validated the interaction between components, including loading body and garment models, applying algorithm configurations, propagating computed distances into aggregated similarity metrics, and ensuring consistency between numerical results and visual output.
- System-level testing evaluated complete analysis workflows, starting from dataset loading through analysis execution to result presentation. This included testing user interactions, configuration changes, repeated execution with identical parameters, and comparison of results across different body-garment combinations.

4.2.3 Test conclusions

The testing process confirmed that the developed system behaves according to its intended design and specification across all defined test levels. Unit-level tests verified the correctness of core geometric algorithms, including marker-based distance computation, hybrid distance selection logic, similarity percentage calculation, and local volume estimation.

Integration tests demonstrated that individual components interact correctly within the analysis pipeline. This includes correct propagation of marker data through distance calculations, consistent application of algorithm configurations, and reliable generation of aggregated comparison results. Special attention was given to numerical stability and repeatability, ensuring that identical inputs and configurations produce consistent outputs across multiple runs.

System-level testing validated complete analysis workflows, covering dataset loading, model parsing, configuration selection, execution of comparison algorithms, and result visualization. Both valid and invalid input scenarios were tested to confirm proper error handling, including missing files, incompatible models, and unsupported configurations.

Statement and decision coverage across the tested execution paths was high, ensuring that all critical conditional branches-such as fallback distance methods, invalid marker handling, and optional enhanced analysis-were executed at least once. This confirms that the system is robust against common edge cases and misuse scenarios.

Overall, the testing results demonstrate that the system is stable, and suitable for its intended use as a research-oriented virtual garment fit analysis tool.

4.2.3.1 Discovered Bugs

No critical defects were discovered during the final testing phase. All tested algorithms executed successfully under both valid and intentionally invalid input conditions. Error handling mechanisms behaved as expected, and no unhandled exceptions or incorrect numerical results were observed during repeated execution.

Minor implementation refinements were addressed during development iterations; however, no unresolved issues affecting correctness, stability, or reproducibility remained at the conclusion of testing. This indicates that the implemented algorithms and data handling logic are sufficiently mature and aligned with the defined system requirements.

4.2.4 Test Runs

```
C:\Users\aleks\Desktop\qualproj\tests>python -m pytest -v
============================================= test session starts =============================================
platform win32 -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0 -- C:\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\aleks\Desktop\qualproj\tests
plugins: dash-3.2.0
collected 17 items

test_analysis_config.py::test_default_configuration PASSED                                          [  5%]
test_analysis_config.py::test_custom_configuration PASSED                                           [ 11%]
test_auth.py::test_login_successful PASSED                                                          [ 17%]
test_auth.py::test_login_unsuccessful PASSED                                                        [ 23%]
test_auth.py::test_admin_privileges PASSED                                                          [ 29%]
test_comparison.py::test_comparison_metrics PASSED                                                  [ 35%]
test_distance_calc.py::test_distance_calculation PASSED                                             [ 41%]
test_distance_calc.py::test_distance_zero PASSED                                                    [ 47%]
test_distance_calc.py::test_multiple_methods_produce_different_results PASSED                       [ 52%]
test_history.py::test_session_saved PASSED                                                          [ 58%]
test_history.py::test_admin_can_delete PASSED                                                       [ 64%]
test_model_loading.py::test_valid_body_model_loaded PASSED                                          [ 70%]
test_model_loading.py::test_valid_garment_model_loaded PASSED                                       [ 76%]
test_model_loading.py::test_invalid_model_rejected PASSED                                           [ 82%]
test_visualization.py::test_heatmap_generation PASSED                                               [ 88%]
test_volume_analysis.py::test_volume_similarity_valid PASSED                                        [ 94%]
test_volume_analysis.py::test_volume_similarity_zero PASSED                                         [100%]
```

Fig 28. Test execution

5. PROJECT MANAGEMENT

5.1 Project organization

The system was designed, implemented, and documented by a single developer as part of a qualification project. The developed software is not a standalone commercial product, but rather a software component within a broader research context, focusing on quantitative garment-body fitting analysis using geometric algorithms.

The core implementation of the project is a Python-based console application intended to be used as a research and analysis tool. For the purposes of the qualification project, additional components such as a graphical user interface and database design were introduced to clearly demonstrate software engineering, system design, and documentation skills. These components serve an illustrative and educational role rather than being strict technical requirements of the underlying research software.

The project was developed in three main phases.

1. **Documentation and System Design**

   The first stage of the project focused on defining the conceptual and structural foundations of the system.

   - The purpose, scope, product vision, and system users were defined.
   - Functional and non-functional requirements were identified based on the intended analytical workflows of the research project.
   - A conceptual database model was created to describe how analytical data, configuration parameters, and results are logically related.
   - A logical and physical database model was then derived to demonstrate relational data design principles, even though persistent storage is not strictly required by the core application.
   - Data flow diagrams, module descriptions, and algorithm flowcharts were prepared to provide a clear overview of system behavior and data movement.

   This phase established a structured blueprint for the system and ensured that further development followed a consistent and well-documented design.

2. **Software Development**

The second stage involved the implementation of the system according to the prepared documentation.

- The core functionality was implemented in Python, focusing on:
  - User authentication logic
  - Model loading and validation
  - Algorithm configuration handling
  - Fit distance calculation
  - Enhanced volume-based analysis
  - Comparison and similarity evaluation
  - Result visualization and reporting
  - Analysis session history management
- The software was developed as a console-based application, reflecting its intended use within a research environment.
- During development, minor adjustments to the documentation were made to reflect implementation details more accurately, such as refinements in algorithm behavior and data structures.

Although the database and graphical interface are not essential for the execution of the research algorithms, they were implemented to demonstrate full-stack design thinking and system integration skills required in a qualification project.

3. **Interface Development and Testing**

In the final stage, additional components were developed to improve clarity, usability, and demonstrability of the system.

- A graphical user interface was created to allow users to:
  - Log in using registered email accounts
  - Upload datasets
  - Configure analysis parameters
  - Execute analyses
  - Visualize results
  - Review historical analysis sessions
- The interface serves as a demonstration layer, making the analytical workflow easier to understand for evaluators and non-technical users.

- A comprehensive testing phase was conducted, focusing on:
    - Correct execution of algorithms
    - Validation of input data
    - Error handling and boundary conditions
    - Role-based access behavior
    - Consistency and correctness of analytical results
- Testing was performed using the pytest framework, and all defined test cases were executed successfully.

Since the project was developed by a single author, testing and validation activities were carried out both manually and programmatically by the developer. In addition, the project manager participated in the validation process by reviewing system behavior and test outcomes. The combined results confirmed that the system operates consistently with the documented requirements and the described algorithmic logic.

5.2 Research

The research phase of the project focused on identifying suitable computational methods, data representations, and software approaches for quantitative evaluation of virtual garment fitting on 3D body models. The goal of the research was not to replicate full-scale commercial simulation systems, but to investigate how objective, geometry-based metrics can be used to compare garment-body relationships in a reproducible and interpretable way.

Several key research directions were explored during the development of the system:

- **3D Body and Garment Representation**

  A core research aspect was the use of polygonal mesh representations for both body models and garment models. Industry-standard formats for 3D meshes were examined to ensure compatibility with existing scanning and modeling pipelines. The research confirmed that triangular mesh structures provide sufficient geometric detail while remaining computationally manageable for distance and volume calculations.

- **Anatomical Marker-Based Analysis**

The use of predefined body markers was researched as a method to localize and standardize geometric measurements across different body models. Marker-based approaches are commonly used in biomechanics and anthropometry, as they allow consistent comparison of distances and spatial relationships across datasets. This concept was adopted to ensure repeatability and comparability of results between different analysis sessions.

- **Distance Calculation Methods**

  Multiple geometric distance computation techniques were reviewed, including nearest-point distance, ray-casting methods, and hybrid approaches that combine several metrics. These methods were evaluated based on numerical stability, computational cost, and suitability for irregular mesh geometries. The system design allows selecting different distance calculation methods in order to compare their influence on the final results.

- **Volume and Similarity Metrics**

  Beyond point-wise distances, research was conducted into volume-based similarity metrics and aggregated similarity scores. Such metrics are frequently used in shape comparison and model validation tasks, as they provide a higher-level quantitative summary of geometric differences. This research informed the implementation of enhanced volume analysis and combined similarity measures in the system.

- **Configuration-Driven Analysis**

  The system was designed to support configurable algorithms, allowing parameters such as distance method, normalization options, and radius factors to be adjusted. This approach was motivated by research practices in computational geometry, where algorithm behavior must be tunable to support experimentation, validation, and reproducibility.

- **Existing Tools and Limitations**

  During the research phase, existing virtual fitting and garment simulation tools were reviewed. Many available solutions rely on proprietary software, closed algorithms, or require complex simulation pipelines that limit transparency and reproducibility. In contrast, the aim of this project was to explore a lightweight, research-oriented approach

that focuses on explicit geometric calculations and interpretable numerical outputs, rather than visually driven or black-box simulation results.

- **Research Context**

  The developed system is part of a broader research context focused on quantitative garment fitting analysis. While the project itself is not intended as a full commercial application, the research conducted provides a foundation for further academic work in digital apparel development, virtual fitting validation, and comparative geometry-based analysis.

5.3 Version control

Version control for this project was implemented using Git, with the source code hosted on GitHub in a dedicated repository: https://github.com/ailves5/garment-fit-analysis
The repository contains:

- Python source code implementing the garment-body fit analysis algorithms
- Supporting modules for model loading, analysis execution, and result comparison
- A lightweight graphical interface created specifically for demonstration purposes
- Documentation and configuration files related to the qualification project

Although the core system was originally designed as a console-based research tool and forms part of a broader research initiative, a graphical user interface and database schema were introduced within the scope of this qualification project to clearly demonstrate software engineering, architectural design, and documentation skills.
Sensitive information such as database credentials is not intended for public exposure in production environments. For the purposes of the qualification project, these values are stored locally and can be adapted to environment variables in future iterations.

Branching and Commit Strategy

Given that the project was developed by a single author, a simplified branching strategy was used:

- The main branch contains the latest stable version of the system.

- Development was performed directly on the main branch, with frequent, incremental commits reflecting logical development steps.

Commits follow a clear and descriptive convention, for example:

- feat: add fit distance calculation module
- feat: implement analysis history management
- refactor: improve model loading structure
- docs: update system documentation

This approach ensures traceability of changes and simplifies rollback if required.

Rollback Procedures

In case it is necessary to revert the project to an earlier stable state, the following Git mechanisms can be used:

1. Checkout a previous commit

```
git checkout <commit_hash>
```

This allows inspection or execution of an earlier version of the system.

2. Create a new branch from an older commit

```
git checkout -b recovery-branch <commit_hash>
```

This approach enables continued development from a historical state without affecting the main branch.

3. Revert commits without rewriting history

```
git revert --no-commit <commit_hash>..HEAD
```

This safely restores the system to a previous state while preserving the commit history.

5.4 Effort estimation

The effort required for the development of this qualification project was estimated based on the actual size of the implemented codebase, the conducted research activities, testing, and documentation work.

```
C:\Users\aleks\Desktop\linesestimation>cloc .
      5 text files.
      5 unique files.
      0 files ignored.

github.com/AlDanial/cloc v 2.06  T=0.10 s (51.7 files/s, 19271.6 lines/s)
-------------------------------------------------------------------------------
Language                     files          blank        comment           code
-------------------------------------------------------------------------------
Python                           5            270            347           1246
-------------------------------------------------------------------------------
SUM:                             5            270            347           1246
-------------------------------------------------------------------------------

C:\Users\aleks\Desktop\linesestimation>
```

Fig 29. Code metrics result

According to the *cloc* analysis, the project consists of 1246 executable lines of code, 347 comment lines, and 270 blank lines, distributed across five source files. Based on commonly accepted software engineering estimation models (e.g., COCOMO and academic benchmarks), a codebase of approximately 1200-1300 lines of Python code corresponds to roughly 2-3 person-months of development effort, depending on algorithmic complexity and domain specificity. Although the total number of lines of code is relatively modest, the project focuses on non-trivial geometric algorithms, data processing pipelines, and configuration-driven analysis, which required significant upfront research and experimentation. At the start of the project, the developer had no prior practical experience in virtual fitting analysis, 3D geometry processing, or garment-body comparison techniques. As a result, a considerable portion of the effort was dedicated to studying domain-specific concepts, learning relevant computational methods, and evaluating different algorithmic approaches before implementation.

A substantial amount of time was therefore spent on selecting appropriate computational strategies, validating algorithmic correctness, and designing a flexible architecture suitable for future extension. In addition to software development, effort was allocated to research, testing, and comprehensive documentation, resulting in a total workload consistent with the initially required three person-months for a qualification project.

Time Period	Category	Activity Description
15/09/2025-28/09/2025	Research	Analysis of virtual fitting concepts, distance-based

		garment evaluation methods, and existing academic approaches
29/09/2025-12/10/2025	Research & Prototyping	Selection and validation of geometric algorithms, early experimental scripts
13/10/2025-7/12/2025	Development	Implementation of core algorithms, model loading, configuration handling, and result computation
8/12/2025-14/12/2025	Testing	Manual and programmatic testing of algorithms, validation of edge cases and error handling
15/12/2025-28/12/2025	Documentation	Writing system description, design diagrams, algorithm explanations, and testing sections
29/12/2025-04/01/2026	Refinement	Final adjustments to documentation, diagrams, effort estimation, and project structure

Table 12. Effort estimation table

APPENDIX

Coding standards

The project is implemented in Python, following general Python best practices inspired by PEP 8 guidelines. These standards emphasize:

- Clear and descriptive naming conventions
- Modular function design
- Explicit parameter passing
- Readable control flow
- Inline comments for non-trivial logic

The goal of the coding style is to ensure that the analytical algorithms are easy to understand, verify, and extend for future research.

Example Code: Central computational components of the system (only hybrid part)

```
def calculate_fit_metrics(self, markers, body_mesh, garment_mesh,
garment_vertices, model_name):
        """Calculate distance from each body marker to the garment surface
using the selected method"""
        distances = {}

        elif self.method == "hybrid":
            # Method 4: Hybrid approach (try multiple methods)
            mesh_center = garment_mesh.bounding_box.centroid
            garment_tree = KDTree(garment_vertices)

            for marker_id, (marker_pos, marker_idx) in markers.items():
                dist = np.nan
                method_used = None

                # Step 1: Normal ray
                normal = body_mesh.vertex_normals[marker_idx]
                locations, _, _ = garment_mesh.ray.intersects_location(
                    ray_origins=[marker_pos],
                    ray_directions=[normal]
                )
                if len(locations) > 0:
                    dists = np.linalg.norm(locations - marker_pos, axis=1)
                    dist = np.min(dists)
                    method_used = "Normal ray"

                # Step 2: Centroid ray
                if np.isnan(dist):
                    direction = mesh_center - marker_pos
                    if np.linalg.norm(direction) > 0:
                        direction /= np.linalg.norm(direction)
```

```
                        locations, _, _ =
garment_mesh.ray.intersects_location(
                            ray_origins=[marker_pos],
                            ray_directions=[direction]
                        )
                        if len(locations) > 0:
                            dists = np.linalg.norm(locations - marker_pos,
axis=1)
                            dist = np.min(dists)
                            method_used = "Centroid ray"

                # Step 3: KDTree
                if np.isnan(dist):
                    dist, _ = garment_tree.query(marker_pos)
                    method_used = "KDTree nearest"

                distances[marker_id] = dist
                self.marker_methods[model_name][marker_id] = method_used

        else:
            raise ValueError(f"Unknown method: {self.method}")

        return distances
```

Table 13. calculate_fit_metrics (only hybrid part) code example

Example Code: Volume analyzer component of the system

```
def analyze_volume_fit(self, markers, body_mesh, garment_mesh, distances,
reference_size=None):
        """Analyze local volume relationships between body and garment."""
        try:
            local_ratios = {}
            local_volumes = {'body': {}, 'garment': {}}
            local_radius = self.get_adjusted_radius(body_mesh,
reference_size)
            for marker_id, (marker_pos, marker_idx) in markers.items():
                if marker_id not in distances or
np.isnan(distances[marker_id]):
                    continue
                body_vol = self.calculate_local_volume(body_mesh,
marker_pos, local_radius)
                garment_vol = self.calculate_local_volume(garment_mesh,
marker_pos, local_radius)
                local_volumes['body'][marker_id] = body_vol
                local_volumes['garment'][marker_id] = garment_vol
                if body_vol > 0:
                    local_ratios[marker_id] = garment_vol / body_vol
                else:
                    local_ratios[marker_id] = np.nan
            valid_ratios = [r for r in local_ratios.values() if not
np.isnan(r)]
            avg_ratio = np.mean(valid_ratios) if valid_ratios else np.nan
            return {
                'local_ratios': local_ratios,
                'local_volumes': local_volumes,
```

```
                'average_volume_ratio': avg_ratio,
                'local_radius': local_radius,
                'radius_factor': self.radius_factor
            }
        except Exception as e:
            if self.verbose:
                print(f"Warning: Local volume analysis failed: {e}")
            return {
                'local_ratios': {},
                'local_volumes': {'body': {}, 'garment': {}},
                'average_volume_ratio': np.nan,
                'local_radius': 0,
                'radius_factor': self.radius_factor
            }
```

Table 14. analyze_volume_fit code example

Coding standard exceptions

- Numerical and geometric calculations prioritize clarity over brevity, leading to longer but more explicit expressions.
- Certain variable names reflect domain-specific terminology (e.g., marker, radius, similarity), which may differ from generic naming conventions.
- Visualization-related code contains domain-specific structures dictated by third-party libraries.

REFERENCES AND SOURCES

[1] C. Palanisamy, N. Anand, M. Tiwari, and J. Suboth, "Development of a web application for 3D body-garment fit analysis," in Proc. 3DBODY.TECH 2024 Conf., Lugano, Switzerland, Oct. 2024. [Online]. doi: 10.15221/24.39.

[2] Y.-L. Lin and M.-J. J. Wang, “The development of a clothing fit evaluation system under virtual environment,” Multimedia Tools and Applications, vol. 75, pp. 7575-7587, 2016, doi: 10.1007/s11042-015-2681-7.

[3] Y. Lu, G. Song, and J. Li, “A novel approach for fit analysis of thermal protective clothing using three-dimensional body scanning,” " Appl. Ergon., vol. 45, no. 6, pp. 1439-1446, Nov. 2014, doi: 10.1016/j.apergo.2014.04.007.

[4] P. R. Apeagyei and R. Otieno, “Usability of pattern customising technology in the achievement and testing of fit for mass customisation,” Journal of Fashion Marketing and Management, vol. 11, no. 3, pp. 349-365, 2007, doi: 10.1108/13612020710763100.

[5] X. Chen, A. Pang, W. Yang, P. Wang, L. Xu, and J. Yu, “TightCap: 3D Human Shape Capture with Clothing Tightness Field,” in ACM Trans. Graph., vol. 1, no. 9, 2021, doi: 10.1145/3478518.

The qualification project ***Comparison system for virtual fitting data used in the garment models' construction process*** was developed at the Faculty of Science and Technology of the University of Latvia, Department of Computing.

With my signature, I confirm that the work was developed independently, only the sources of information specified in it were used and that the submitted electronic copy of the submitted work corresponds to the printout and/or the copy presented to the opponent.

Author: ***Aleks Ilves*** _____ **5.01.2026**.

I recommend the paper for defense
Advisor: ***Dipl.-Math. (WWU Münster) Andrejs Neimanis*** _____ **5.01.2026.**

Opponent: <***scientific degree, first name, surname***> _____

Paper submitted **5.01.2026.**
Person authorized by the dean (electronic signature)